

Advanced NLP

Session 14 - Vision-Language Models (VLM: CLIP, LLaVA) & Multimodality

Teacher: Dr. Behrouz Minaei-Bidgoli
Presenter: Poorya Saneei
Presentation creator: Poorya Saneei

Iran University of Science & Technology

Contents

1. Introduction
2. Multi-to-text
3. Multi-to-image / Multi-to-Multi

1.

Introduction

Multimodal Modeling Paradigms

Text-to-text

Text — Model — Text

Multi-to-text

TODAY'S LECTURE

Text
Image
— Model —
Text

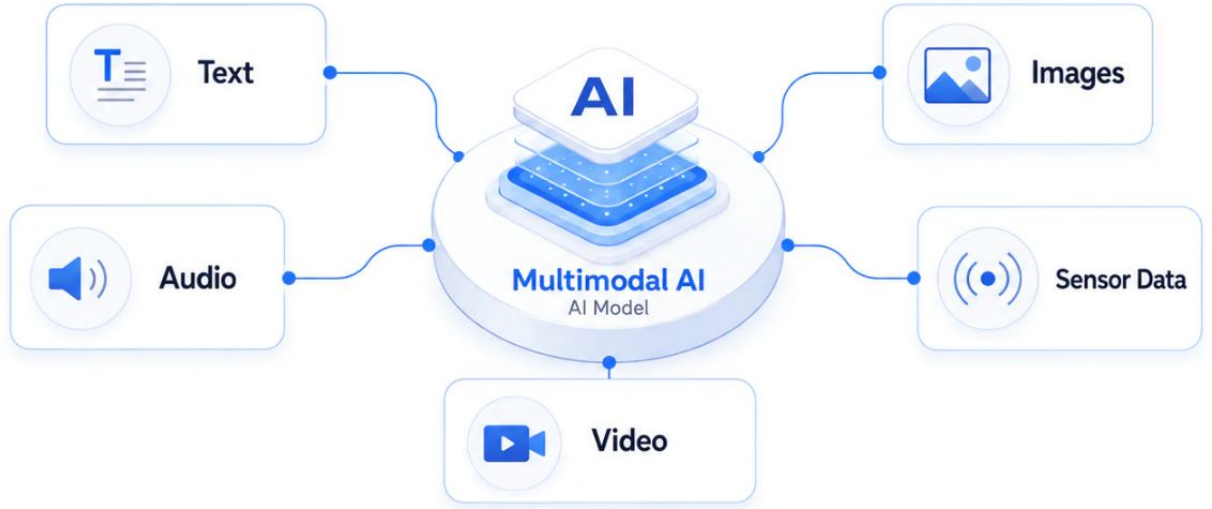
Multi-to-image

Text
Image
— Model —
Image

Multi-to-multi

Text
Image
— Model —
Text
Image

Multimodal Application Examples



1 Traditional AI usually works with one data type at a time.

2 **Multimodal AI** combines multiple modalities simultaneously for richer, more nuanced understanding.

Better context
Understands the full picture across different data types.

Richer perception
Captures more signals from the real world.

More accurate insights
Delivers deeper, more reliable decisions and predictions.

Key Real-World Multimodal Applications

E-Commerce & Retail

Visual Search & Try-On

Integrates image recognition with text queries to power modern shopping experiences.

Key Use Cases:

- Snap-to-search product discovery
- Virtual AR try-on for apparel & cosmetics
- Cross-modal recommendations

Healthcare & Diagnostics

Medical Imaging QA

Combines clinical radiological scans (X-rays, CT) with patient EHR text history.

Key Use Cases:

- Automated radiology report synthesis
- Visual diagnostic assistance
- Multi-modal patient triage

Autonomous Systems

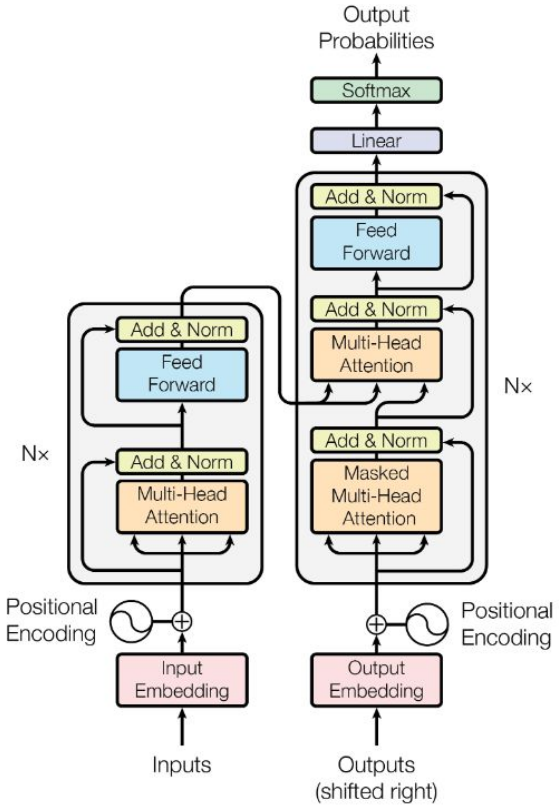
Web & Embodied Agents

Fuses DOM text, visual UI snapshots, and sensor feeds to execute real-world tasks.

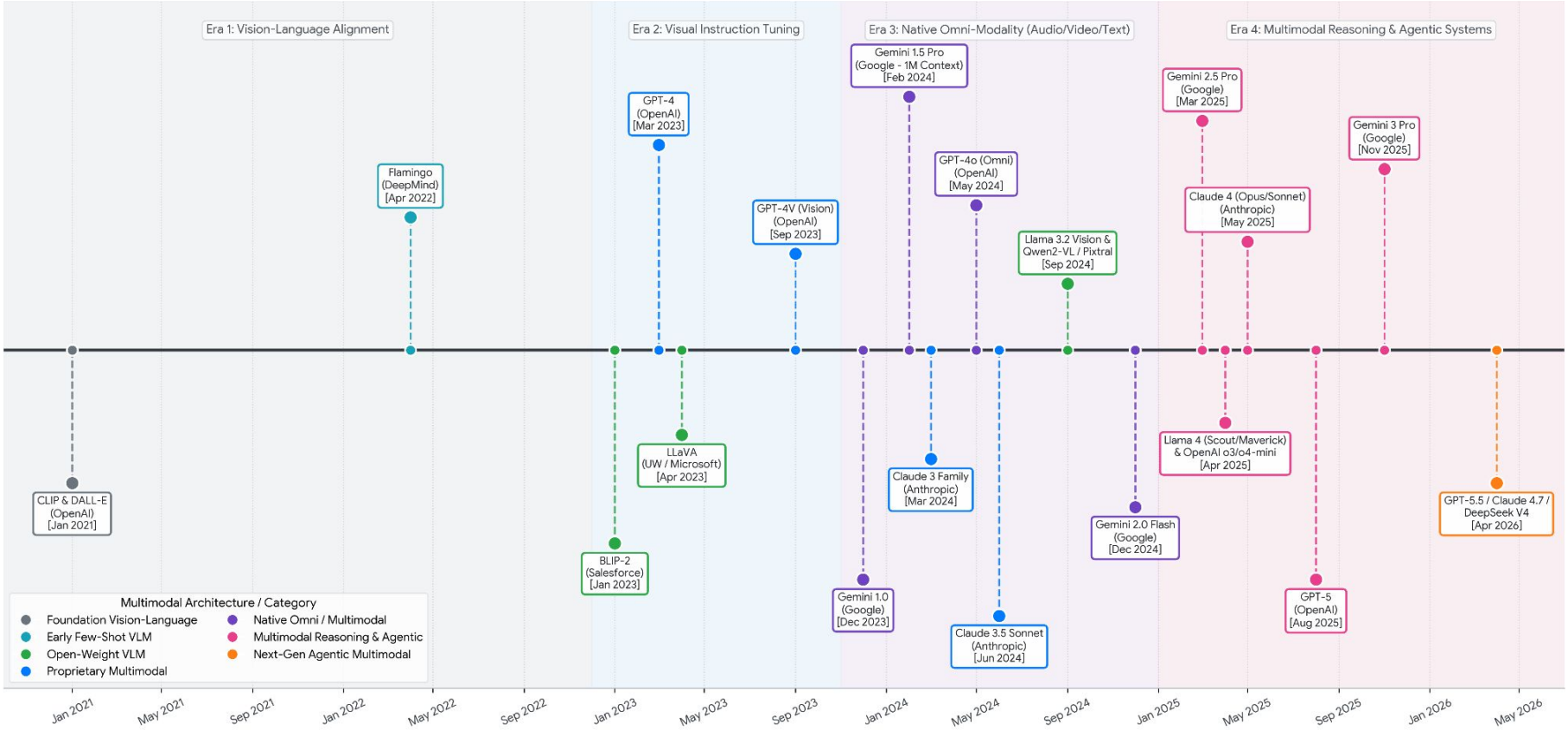
Key Use Cases:

- Autonomous GUI navigation
- Robotic visual grounding & action
- Real-time video & sensor analysis

Attention Is All You Need



Historical Release Timeline of MMLLM



2.

Multi-to-text

Main Resource For this Section: https://cmu-l3.github.io/anlp-spring2026/

Multi-to-text

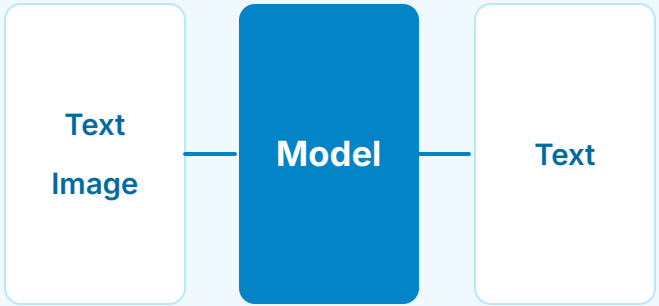
Topic 01
Vision Architecture Basics
ViT [Dosovitskiy et al. 2020] — Vision Transformers

Topic 02
Learning Image Representations
CLIP [Radford et al. 2021] — Contrastive Pre-training

Topic 03
Combining with a Language Model
LLaVA [Liu et al. 2023] — Visual Instruction Tuning

Multi-to-text

TODAY'S LECTURE



Focuses on models processing multimodal inputs (text + vision) to generate textual outputs.

Vision Transformer (ViT)

Published as a conference paper at ICLR 2021

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy[*,†], **Lucas Beyer**[*], **Alexander Kolesnikov**[*], **Dirk Weissenborn**[*], **Xiaohua Zhai**[*], **Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby**[*,†]

[*]equal technical contribution, [†]equal advising

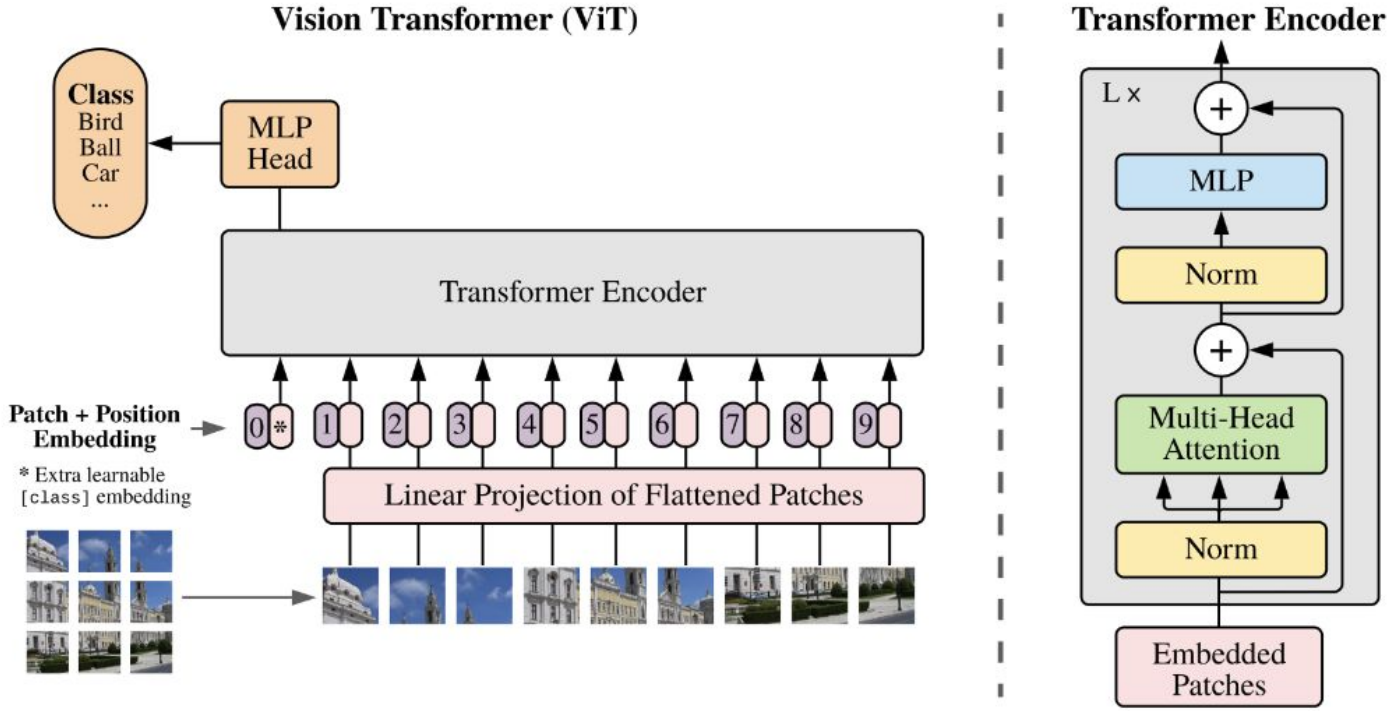
Google Research, Brain Team

{adosovitskiy, neilhoulsby}@google.com

Vision Transformer (ViT)

THE CORE IDEA

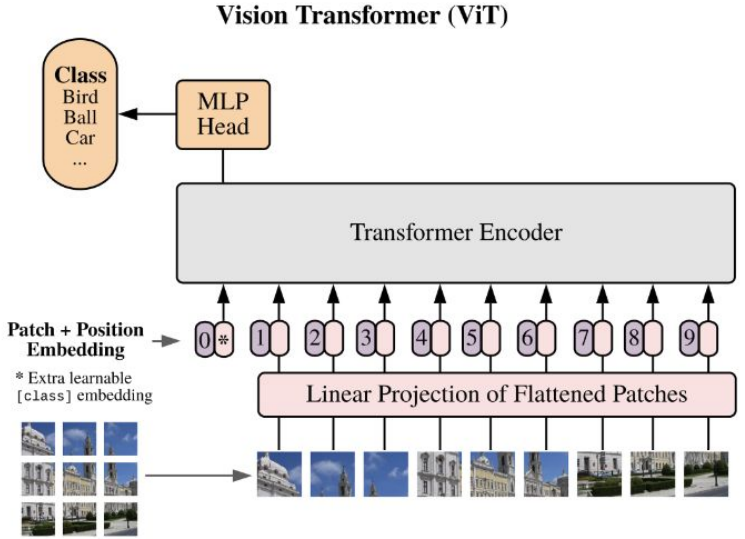
Standard Transformers for Vision - Divide an image into patches, flatten patches into vectors, and process them like standard text token embeddings.

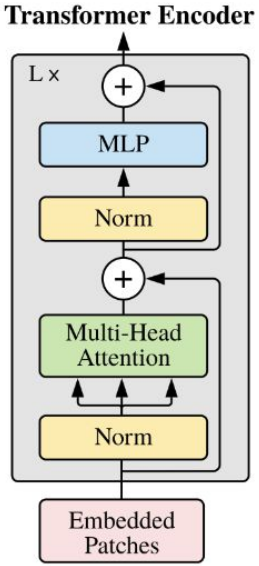


Vision Transformer (ViT)

THE CORE IDEA

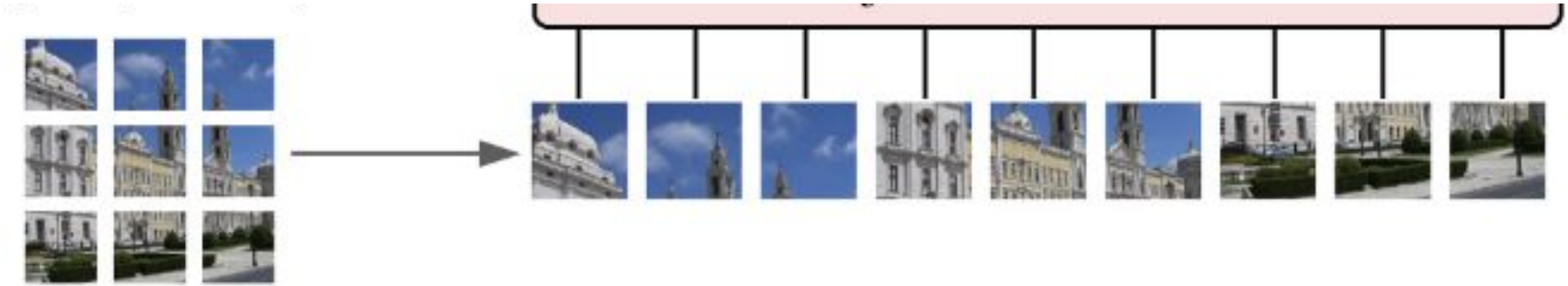
Standard Transformers for Vision - Divide an image into patches, flatten patches into vectors, and process them like standard text token embeddings.



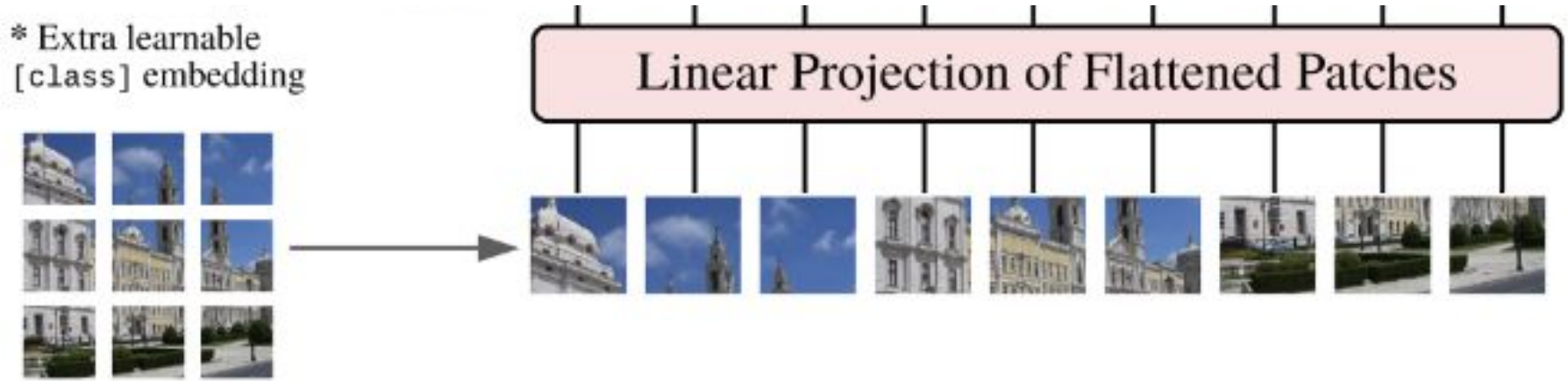


- Divide an input image into 196 (14x14) small images of size (16x16)
- Treat it as embedding in NLP
- Use it as an input for traditional transformer encoder (like in BERT)
- Use 12 transformer layers (Norm, Multi-head attention, etc.)
- Take the last output, use it as input for Dense Layer with 1000 classes
- Voilà - you have a classification model

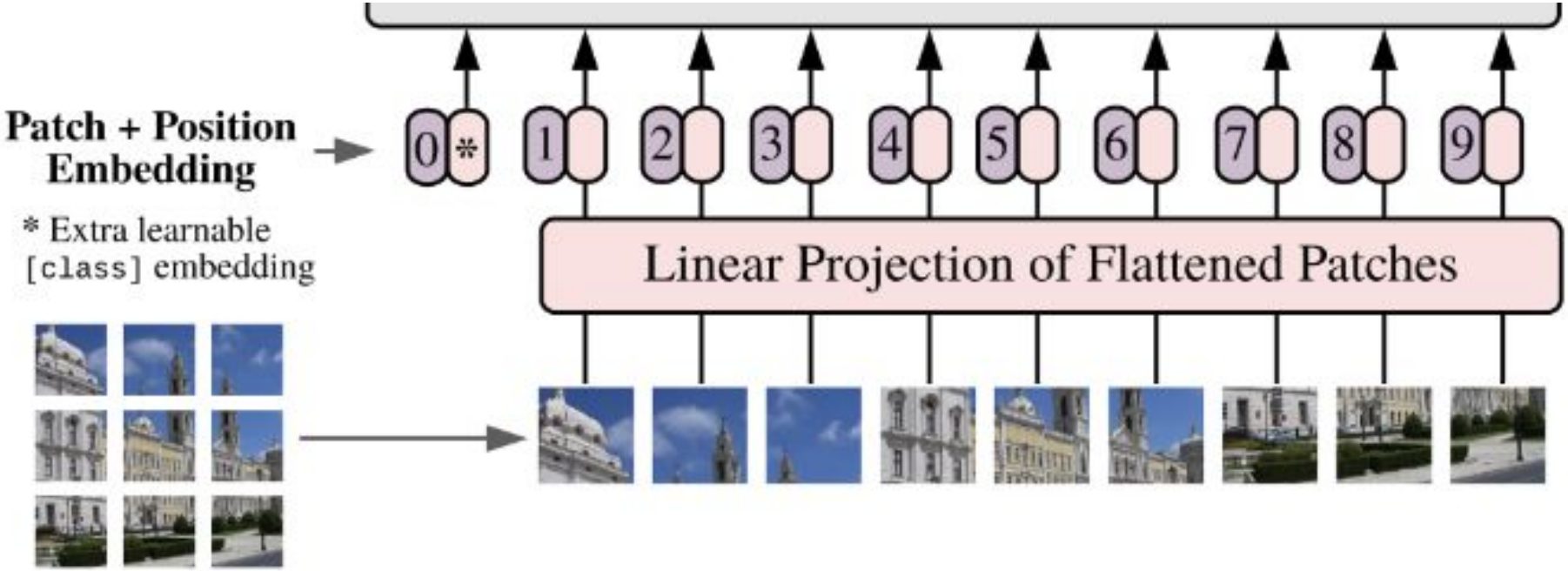
Vision Transformer (ViT)



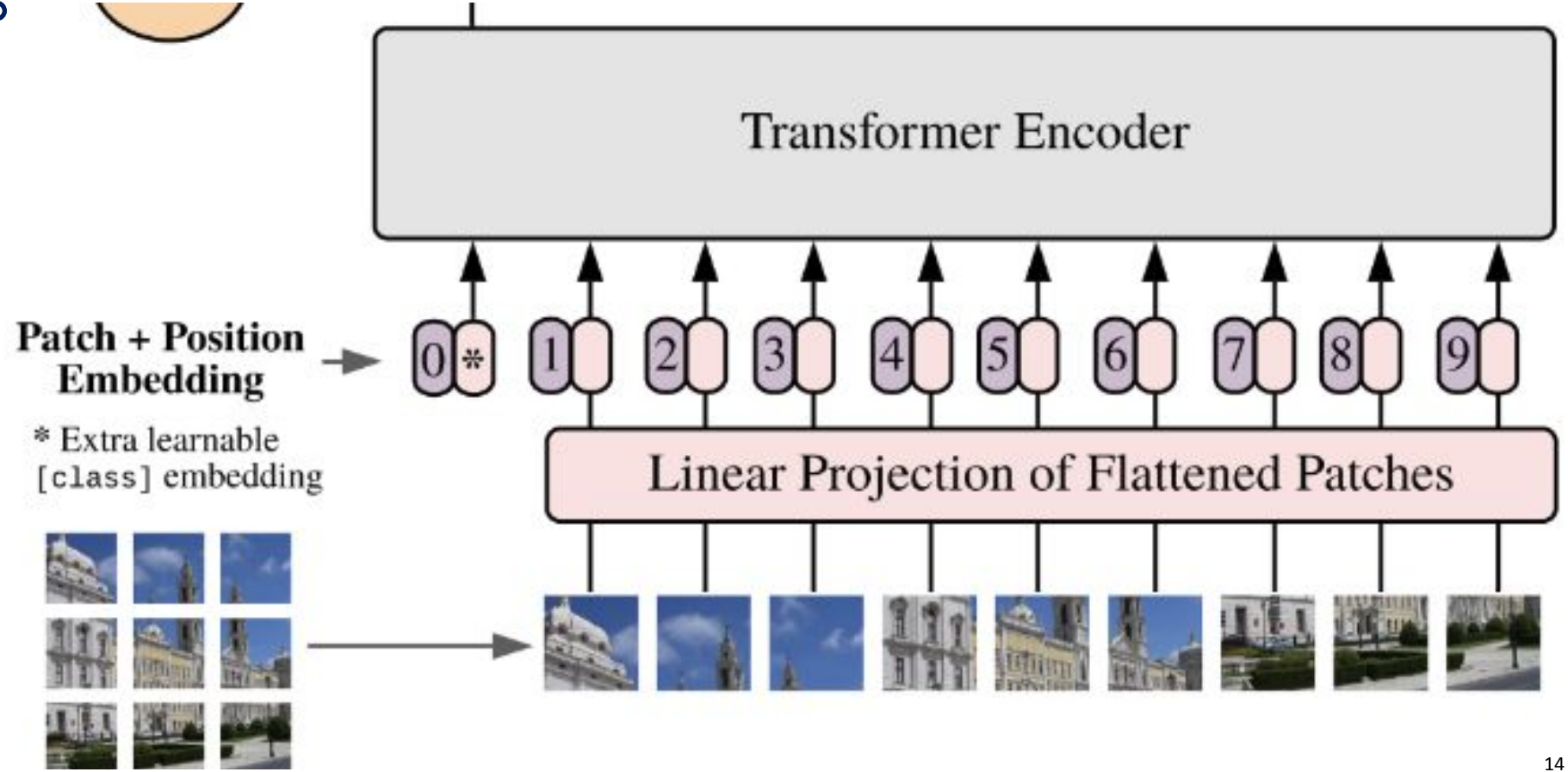
Vision Transformer (ViT)



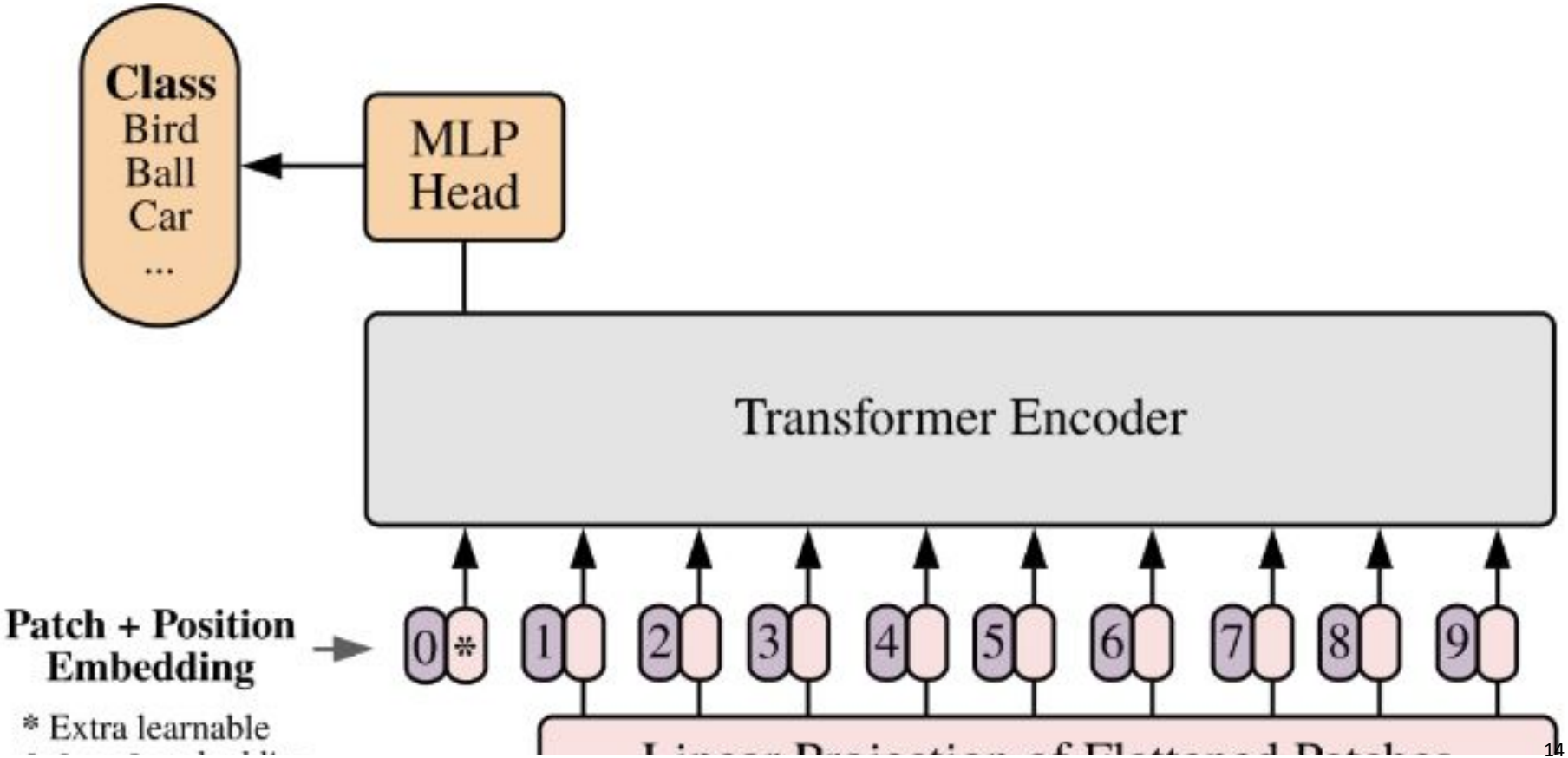
Vision Transformer (ViT)



Vision Transformer (ViT)



Vision Transformer (ViT)



Vision Transformer (ViT)

Vision Transformer (ViT)

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1\mathbf{E};\ \mathbf{x}_p^2\mathbf{E};\ \cdots\ ;\ \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{pos}, \qquad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D},\ \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D} \tag{1}$$

$$\mathbf{z'}_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \qquad \ell = 1 \ldots L \tag{2}$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z'}_\ell)) + \mathbf{z'}_\ell, \qquad \ell = 1 \ldots L \tag{3}$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0) \tag{4}$$

Vision Transformer (ViT)

01. PARAMETER EXPLOSION

Why Not Full Images?

COMPUTATIONAL COMPLEXITY

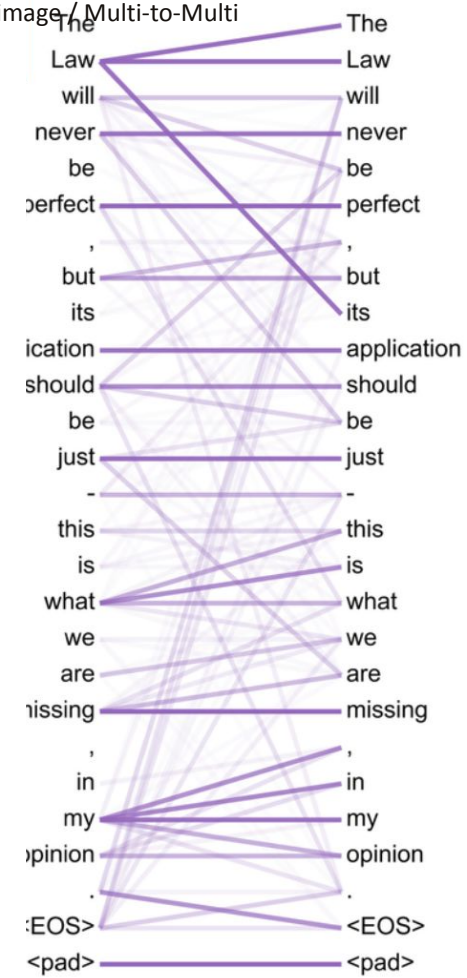
$$(3 \times 224 \times 224)^2 = 22B$$

Quadratic Memory Scaling

Standard self-attention requires storing n^2 pairwise interactions between all pixels.

Hardware Constraint

22 billion parameters per layer exceeds standard GPU memory limits.



Vision Transformer (ViT)

PATCH EMBEDDINGS & MATHEMATICAL FORMULATION

Image Patching & Dimensions – Flattening 2D images into sequences of patch vectors for transformer input.

PATCH EMBEDDINGS

Image Input Size

(3 × 224 × 224)

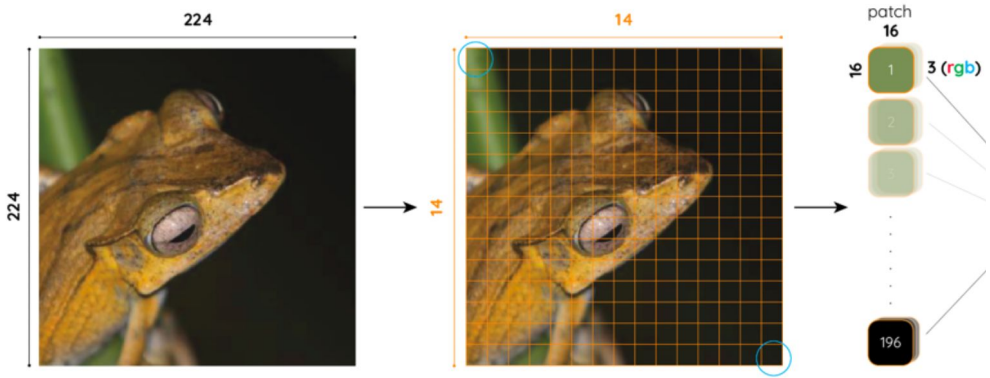
Patch Division

196 Patches (14 × 14)

Patch Dimensions: 3 × 16 × 16

Spatial Formula

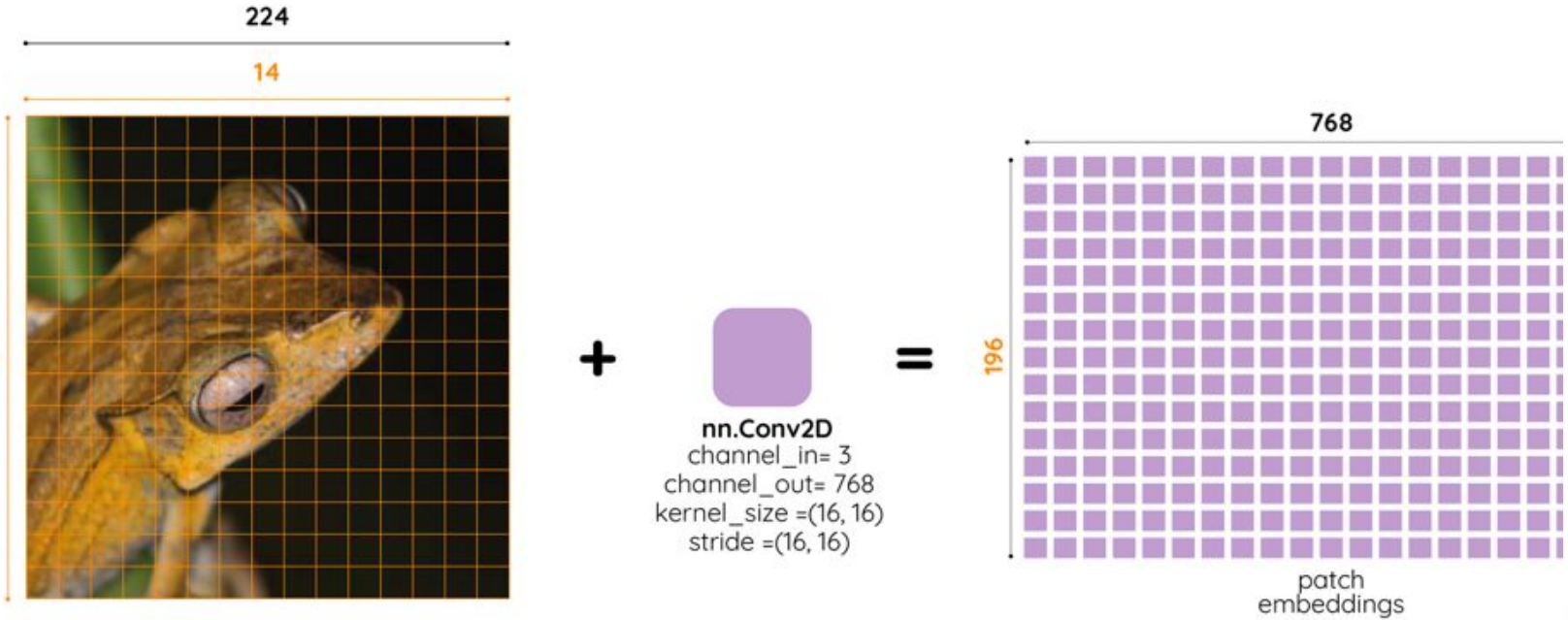
16 × 14 = 224 (original image size)



Vision Transformer (ViT)

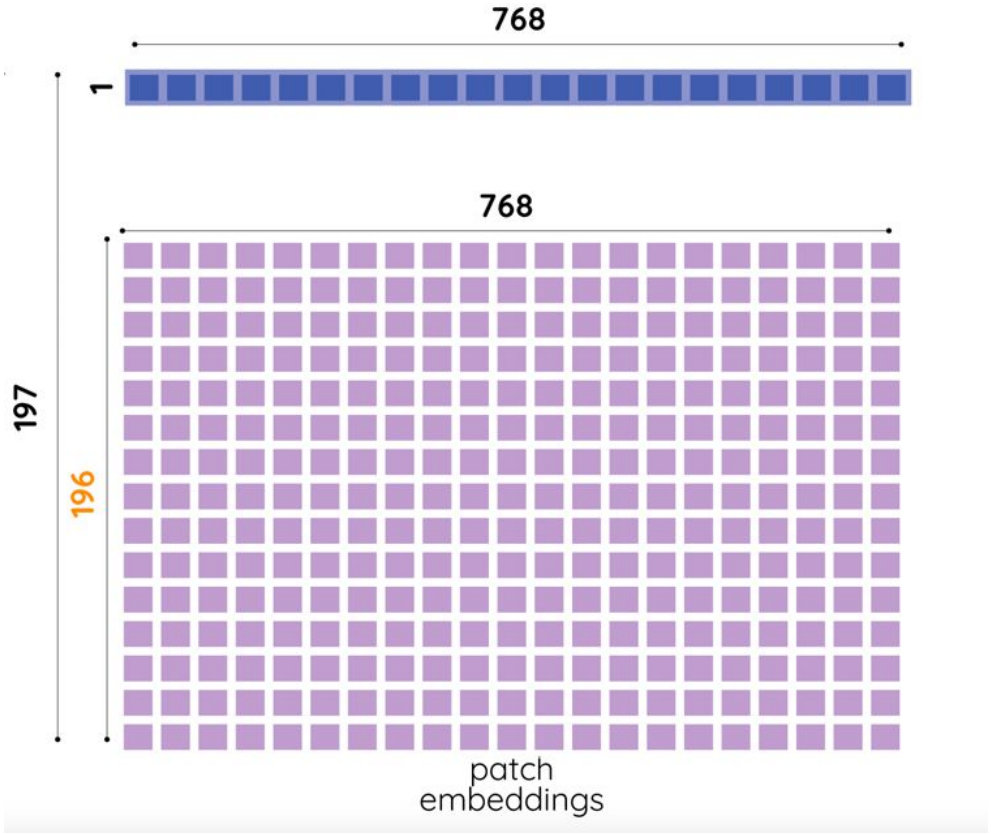
PATCH EMBEDDINGS & MATHEMATICAL FORMULATION

Image Patching & Dimensions – Flattening 2D images into sequences of patch vectors for transformer input.



Vision Transformer (ViT)

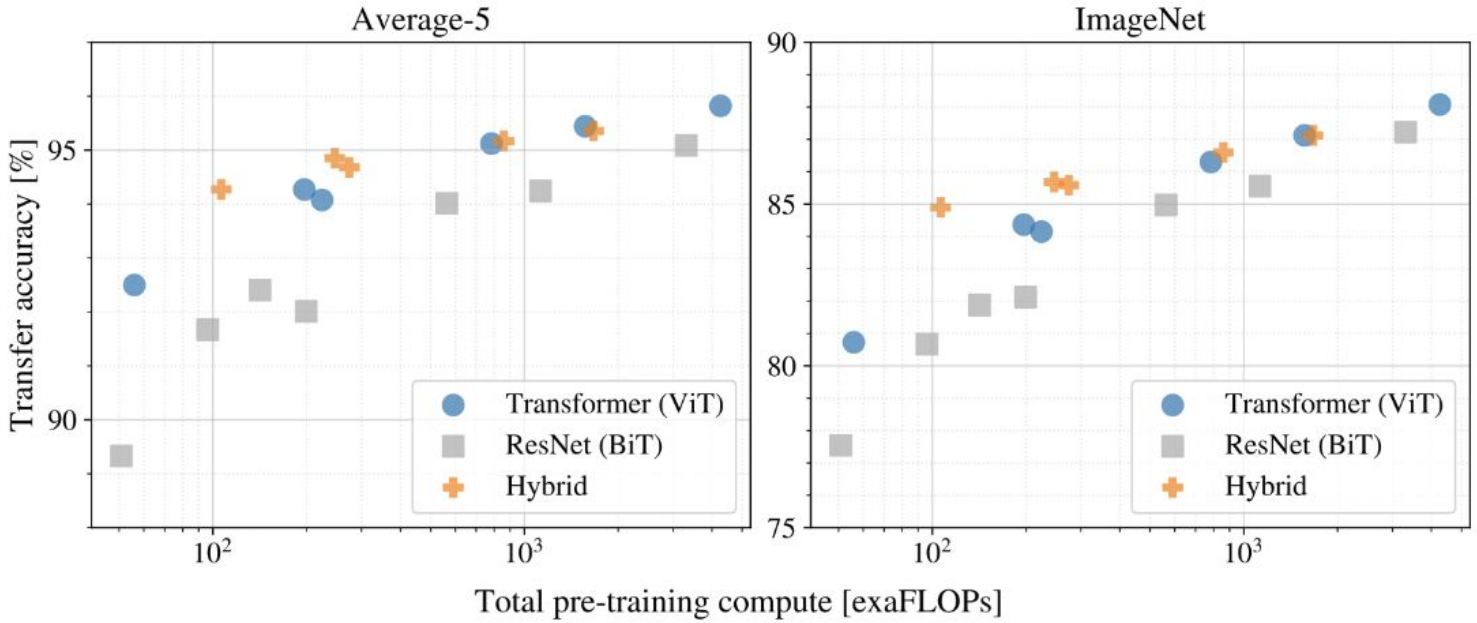
[CLS] Token



Vision Transformer (ViT)

PERFORMANCE & COMPUTE EFFICIENCY

Performance vs. Pre-training Compute – Vision Transformers attain superior transfer accuracy when scaled with large pre-training compute.



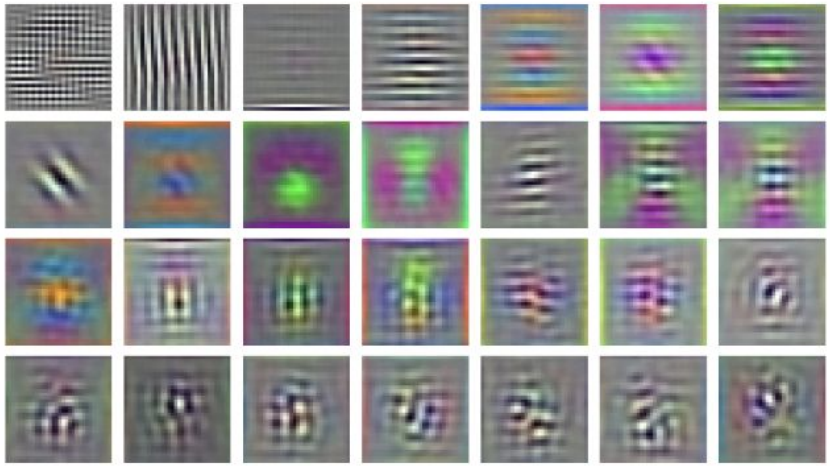
Performance versus pre-training compute for different architectures

Vision Transformer (ViT)

RGB EMBEDDING FILTERS & PROJECTION

Linear Projection & Filter Visualization – The linear projection layer learns plausible basis functions for encoding image patches.

RGB embedding filters
(first 28 principal components)



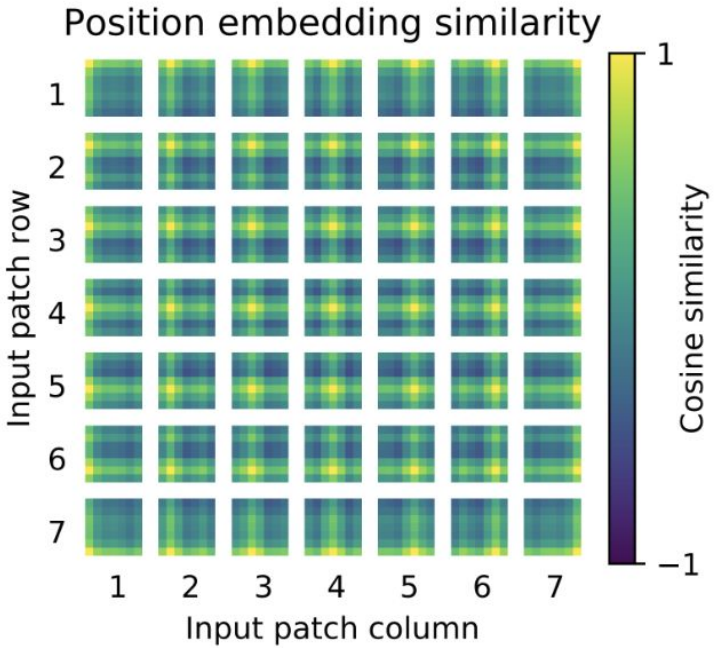
$$x = Wx_p,\ W_e \in \mathbb{R}^{D\times(P^2\cdot C)}$$

Reshape rows into P x P, visualize principal components

Vision Transformer (ViT)

POSITION EMBEDDING SIMILARITY

Position Embedding Similarity – Cosine similarity between the position embedding of the patch with the indicated row and column and the position embeddings of all other patches.



Cosine similarity between the position embedding of the patch with the indicated row and column and the position embeddings of all other patches

Vision Transformer (ViT)

ATTENTION MAPS & SALIENCY

Self-Attention Visualizations – The model attends to image regions that are salient for the target task (e.g., classification).

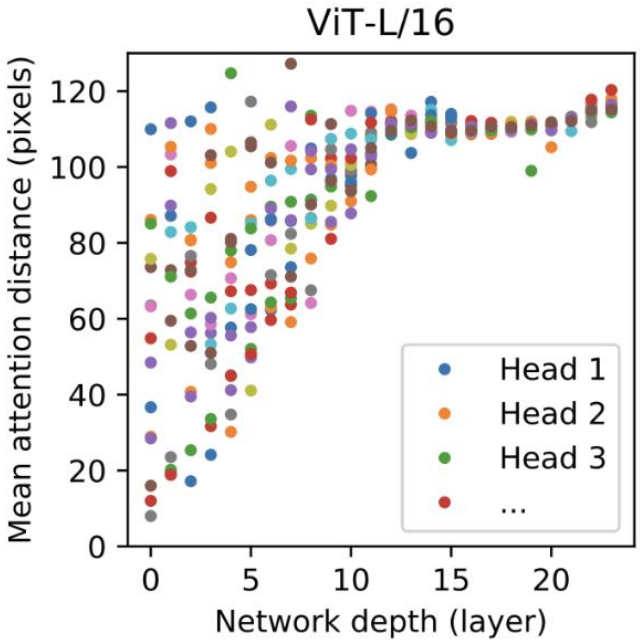


Can attend to regions that are salient for the task (here classification)

Vision Transformer (ViT)

MEAN ATTENTION DISTANCE

Attention Distance across Layers – Early layers either attend to large regions or narrow regions; later layers generally attend to larger regions.



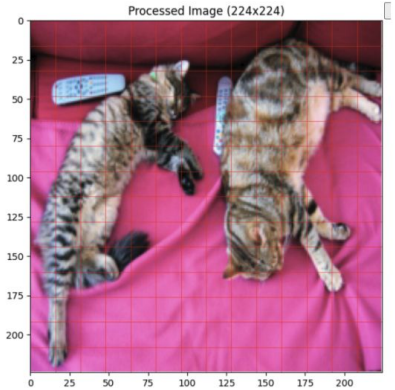
Early layers either attend to large regions or narrow regions; later layers generally attend to larger regions

Vision Transformer (ViT)

CODE EXAMPLE & PIPELINE

ViT Inference & Architecture – Step-by-step patch embedding, model parameters, and top output predictions.

Code example




```
Patch embedding process:
1. Input image: torch.Size([1, 3, 224, 224])
   [batch, channels=3, height=224, width=224]

2. After patch projection: torch.Size([1, 196, 768])
   [batch, num_patches=196, hidden_size=768]

3. Each 16x16x3 patch → 768-dim vector
```

```
Model architecture:
Number of layers: 12
Number of attention heads: 12
Hidden size: 768
Head dimension: 64

Output shape: torch.Size([1, 197, 768])

CLS token representation shape: torch.Size([1, 768])
  (Used for classification tasks)
```

```
Top 5 predictions:
========================================
1. Egyptian cat: 93.74%
2. tabby, tabby cat: 3.84%
3. tiger cat: 1.44%
4. lynx, catamount: 0.33%
5. Siamese cat, Siamese: 0.07%
```

https://github.com/cmu-l3/anlp-spring2026-code

Patch Embedding Process

1. Input: `torch.Size([1, 3, 224, 224])`
2. Projected: `torch.Size([1, 196, 768])`

Each 16×16×3 patch → 768-dim vector

Model Architecture

Layers: **12** | Heads: **12** | Hidden: **768** | Head Dim: **64**

Output shape: `torch.Size([1, 197, 768])`

CLS token: `torch.Size([1, 768])` (for classification)

Top 5 Predictions

1. Egyptian cat: 93.74%

2. Tabby, tabby cat: 3.84% | 3. Tiger cat: 1.44%

4. Lynx, catamount: 0.33% | 5. Siamese cat: 0.07%

https://github.com/cmu-l3/anlp-spring2026-code

Vision Transformer (ViT)

PERFORMANCE & SCALING LAWS

How Good is the Vision Transformer? – Evaluating performance dynamics across dataset scale and computational efficiency.

Small / Medium Datasets

e.g., ImageNet-1k

Trained from Scratch

Underperforms standard CNNs (like ResNet) when trained from scratch.

Why? (Inductive Bias)

Lacks CNN inductive biases such as locality and translation equivariance.

Massive Datasets

e.g., ImageNet-21k, JFT-300M

High-Capacity Scaling

Scales remarkably well as dataset size expands into millions of images.

Data Compensation

Massive data volume effectively compensates for the lack of built-in inductive bias.

The Takeaway

Pre-trained Checkpoints

SOTA Performance

Pre-trained ViT outperforms state-of-the-art, much larger CNN architectures.

Compute Efficiency

Requires substantially less computational cost for fine-tuning on downstream tasks.

Vision Transformer (ViT)

COMPUTE COST & CRITICISMS

The Reality Check – Should You Train ViT from Scratch?

Price Tag of Scale

HIGH COMPUTE DEMAND

~2,500 TPUv3 Core-Days

Required to train state-of-the-art ViT top models from scratch.

ESTIMATED FINANCIAL COST

~$30,000 / Training Run

Calculated at standard cloud rates (~$0.50 per core-hour).

Main Criticisms

Data Hungry

Superior accuracy is locked behind massive, often proprietary datasets.

Prohibitive Cost

Pre-training from scratch is only feasible for Big Tech (FAANG) or mega labs.

Architectural Debate

Is linear patch projection fundamentally different from strided convolutions?

Rule of Thumb

RECOMMENDED STRATEGY

Use Pre-trained Checkpoints

Fine-tune existing models unless you have massive compute capabilities.

PRACTICAL ALTERNATIVES

EfficientNet / ConvNeXt

Much more practical choices for standard budgets and training setups.

Vision Transformer (ViT)

PARADIGM SHIFT & IMPACT

Paradigm Shift & Impact on Computer Vision – From Pure Attention to Hierarchical Architectures

CNN Monopoly Broken

PURE ATTENTION WINS

First time pure Attention models beat CNNs in core vision tasks, sparking an explosion of Vision Transformers in 2021+.

KEY SHIFT

Demonstrated that standard Transformer architectures could surpass specialized visual inductive biases at scale.

New Naming Trend

"X IS WORTH Y" TITLES

Inspired a widespread wave of paper titles across image and video domains:

- **"An Image is Worth 16×16 Words"**
- **"A Video is Worth Three Views"**

Architectural Evolution

HIERARCHIES & LOCALITY

Researchers quickly addressed ViT's limitations by re-introducing multi-scale hierarchies and local windows.

KEY MILESTONE

Swin Transformer *(Liu et al., 2021)*

Brought shifted windows and linear computational complexity to vision transformers.

Vision Transformer (ViT)

FUTURE TRAJECTORIES & UNLOCKED POTENTIAL

The Road Ahead (What ViT Unlocked) – Key Trajectories for Transformers in Vision

01. MULTIMODAL

Unified Multimodal Modeling

Since Transformers are natively sequence-to-sequence models, bridging vision and language (Image Captioning, VLMs) becomes seamless.

02. EMBEDDINGS

Off-the-Shelf Embeddings

BERT-style pre-trained visual representations that drastically accelerate downstream CV prototyping.

03. SCALING

Extreme Scaling

Pushing model parameters and dataset sizes even further for zero-shot and few-shot generalization.

04. TOKENIZATION

Smarter Patch Tokenization

Moving beyond rigid 16×16 grids toward more dynamic, efficient patch extraction and token pruning methods.

Vision Transformer (ViT)

Published as a conference paper at ICLR 2021

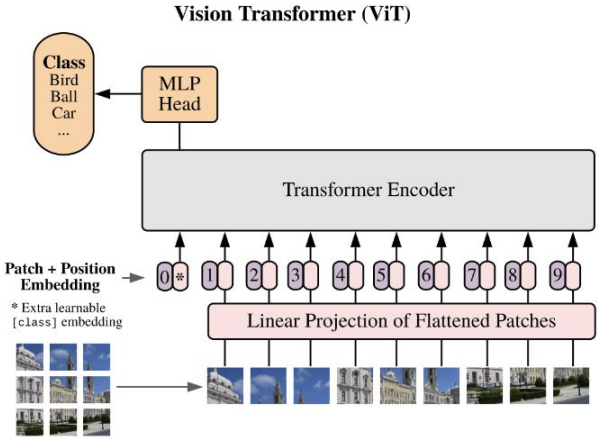
AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy[*,†], **Lucas Beyer**[*], **Alexander Kolesnikov**[*], **Dirk Weissenborn**[*], **Xiaohua Zhai**[*], **Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby**[*,†]

[*]equal technical contribution, [†]equal advising

Google Research, Brain Team

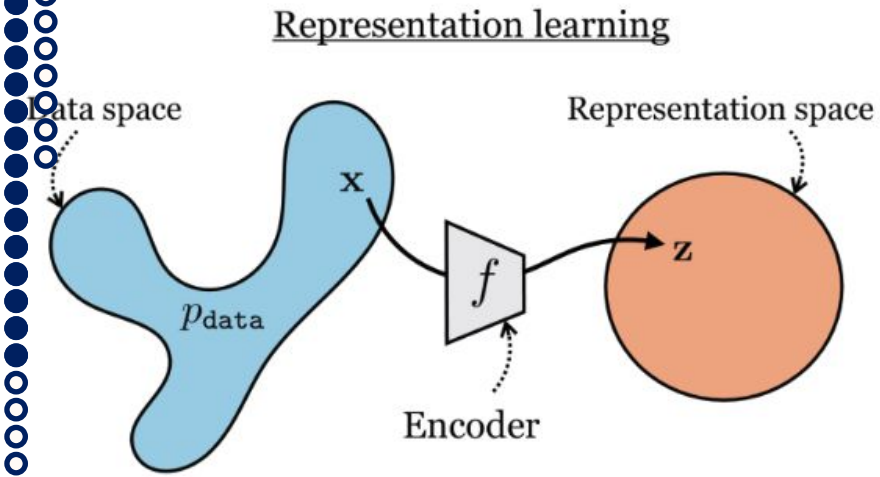
{adosovitskiy, neilhoulsby}@google.com

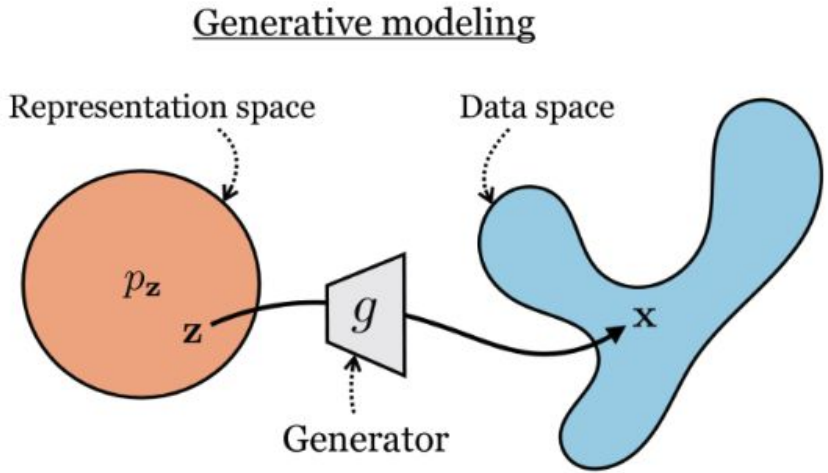


REPRESENTATION LEARNING & GENERATIVE MODELING

Learning Image Representations

Mapping between high-dimensional data space (x) and compact representation space (z)





- Primary paradigm for classification, search, & retrieval.

`Mapping: f(x) → z`

- Goal: Extract compact, meaningful semantic features.
- Encodes high-dimensional visual data **pdata** into latent vector **z**.

- Key paradigms: VAEs, GANs, Diffusion, Autoregressive.

`Mapping: g(z) → x`

- Goal: Synthesize realistic new image samples.
- Decodes latent code **z ~ pz** back to data space **x**.

REPRESENTATION LEARNING & GENERATIVE MODELING

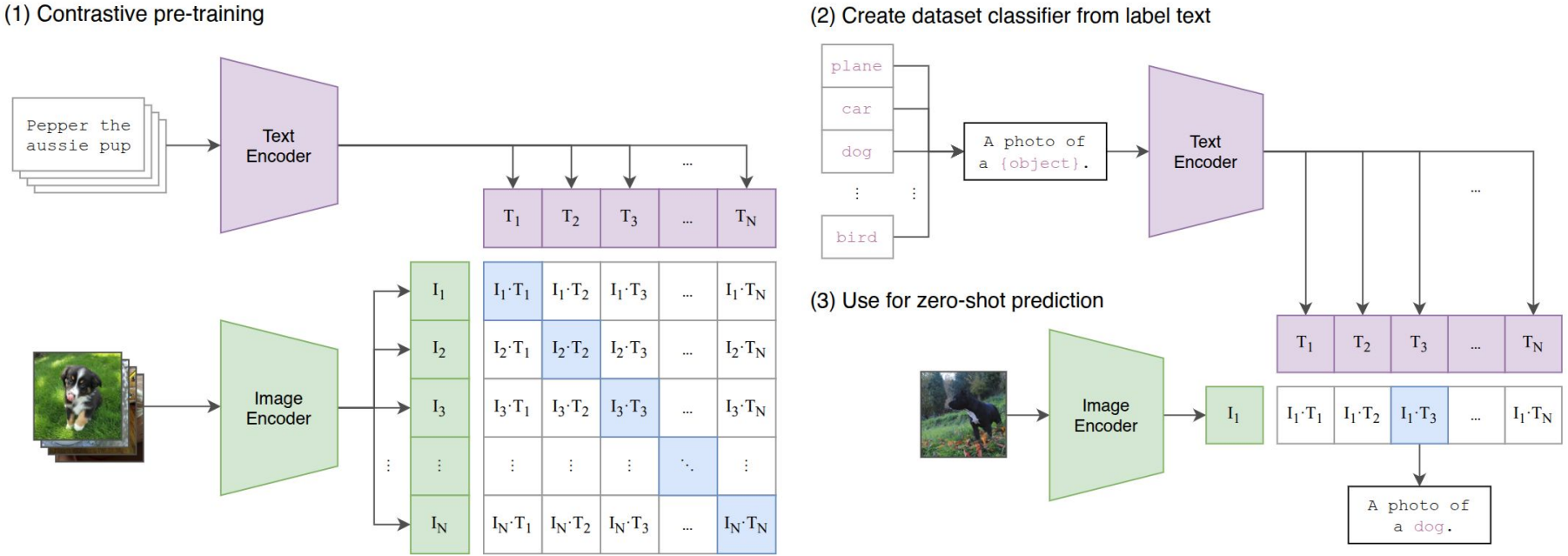
Learning Image Representations

Learning Transferable Visual Models From Natural Language Supervision

Alec Radford [*1] Jong Wook Kim [*1] Chris Hallacy [1] Aditya Ramesh [1] Gabriel Goh [1] Sandhini Agarwal [1]
Girish Sastry [1] Amanda Askell [1] Pamela Mishkin [1] Jack Clark [1] Gretchen Krueger [1] Ilya Sutskever [1]

REPRESENTATION LEARNING & GENERATIVE MODELING

Learning Image Representations



MULTIMODAL REPRESENTATION LEARNING

Contrastive Language–Image Pre-training (CLIP)

Basic Idea: Joint Contrastive Pair Classification

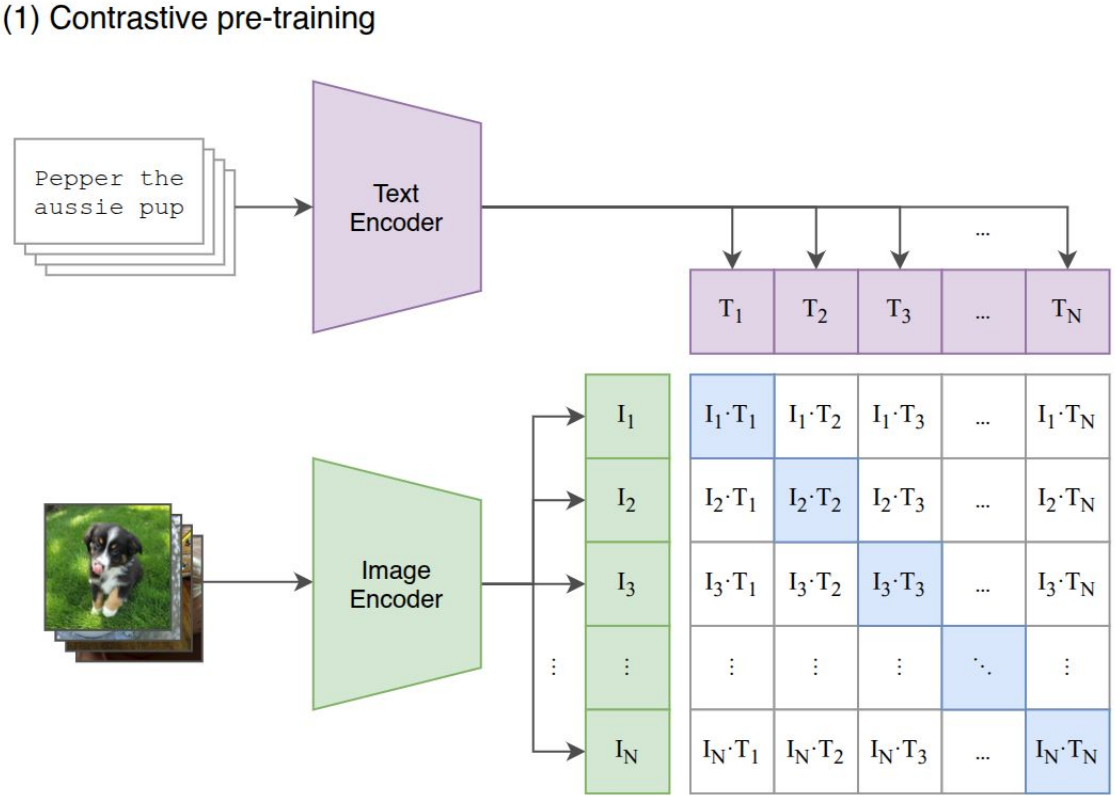
01. CORE OBJECTIVE

Pairwise Matching

N (Image, Text) Pairs
Given a batch of N paired images and text descriptions.

N × N Matrix
Predict which image is paired with which text across all N^2 combinations.

Contrastive Goal
Maximize diagonal scores $(I_\square \cdot T_\square)$ while minimizing all off-diagonal scores.



MULTIMODAL REPRESENTATION LEARNING

Contrastive Language–Image Pre-training (CLIP)

Loss Function: Symmetric Cross-Entropy Objective

$$L((x_1, y_1), \ldots, (x_N, y_N)) =$$

Push up dot product of the correct pair

$$-\frac{1}{2}\sum_{n=1}^{N}\left[\log\frac{\exp\left(f_I(x_n)^\top f_T(y_n)\right)}{\sum_j \exp\left(f_I(x_j)^\top f_T(y_n)\right)} + \log\frac{\exp\left(f_I(x_n)^\top f_T(y_n)\right)}{\sum_j \exp\left(f_I(x_n)^\top f_T(y_j)\right)}\right]$$

Push down dot product of other pairs

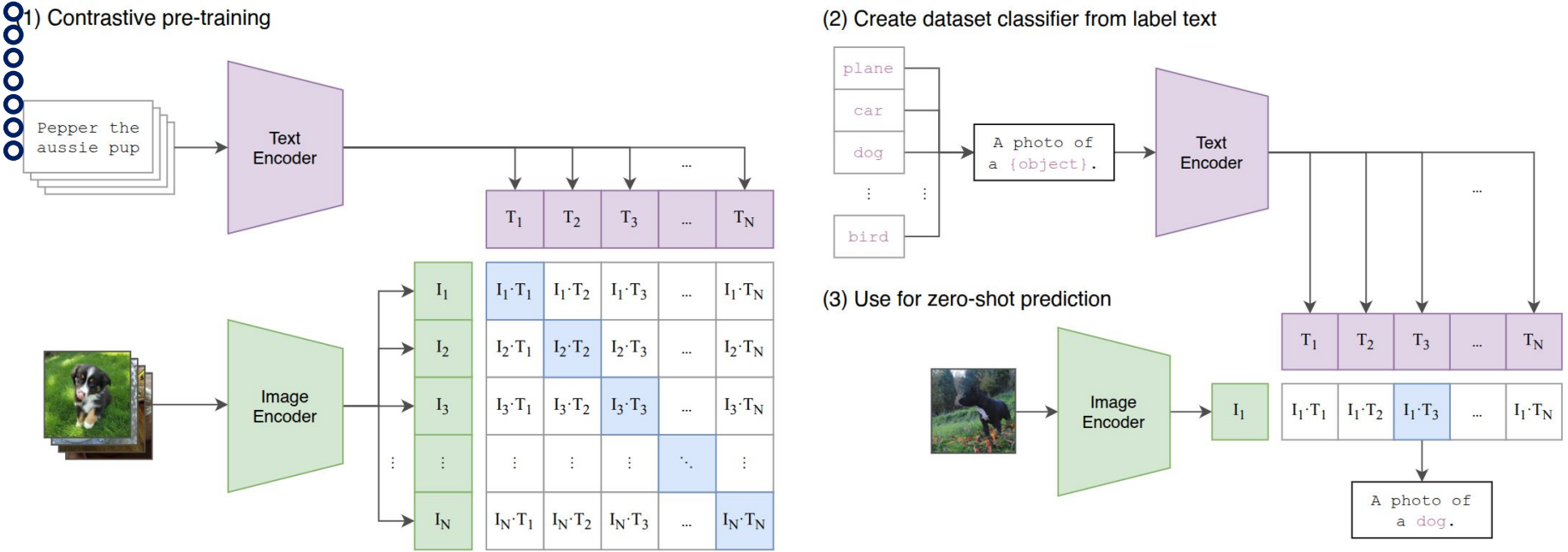
Softmax over images

Softmax over text

Each term is a cross-entropy loss, where $p_\theta \propto f_I(x)^\top f_T(y)$ and p_* puts all of its mass on the correct pair

REPRESENTATION LEARNING & GENERATIVE MODELING

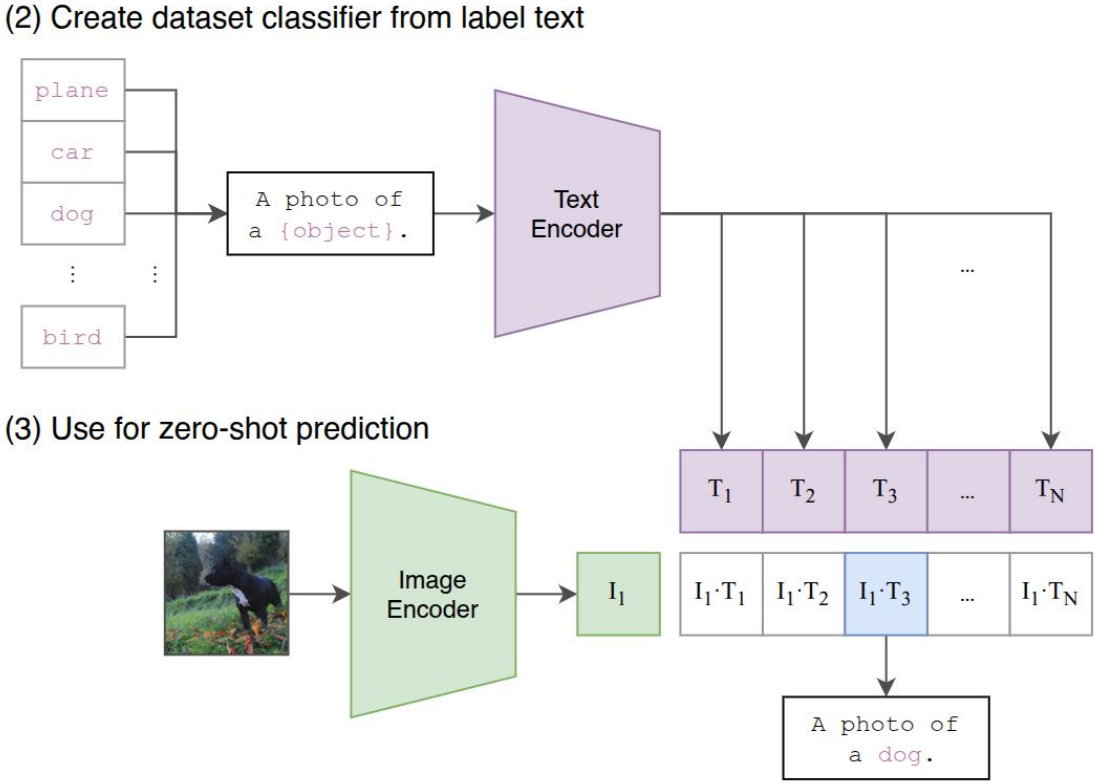
Learning Image Representations



MULTIMODAL REPRESENTATION LEARNING

Contrastive Language–Image Pre-training (CLIP)

Zero-Shot Prediction: Dataset Classifier Construction & Inference

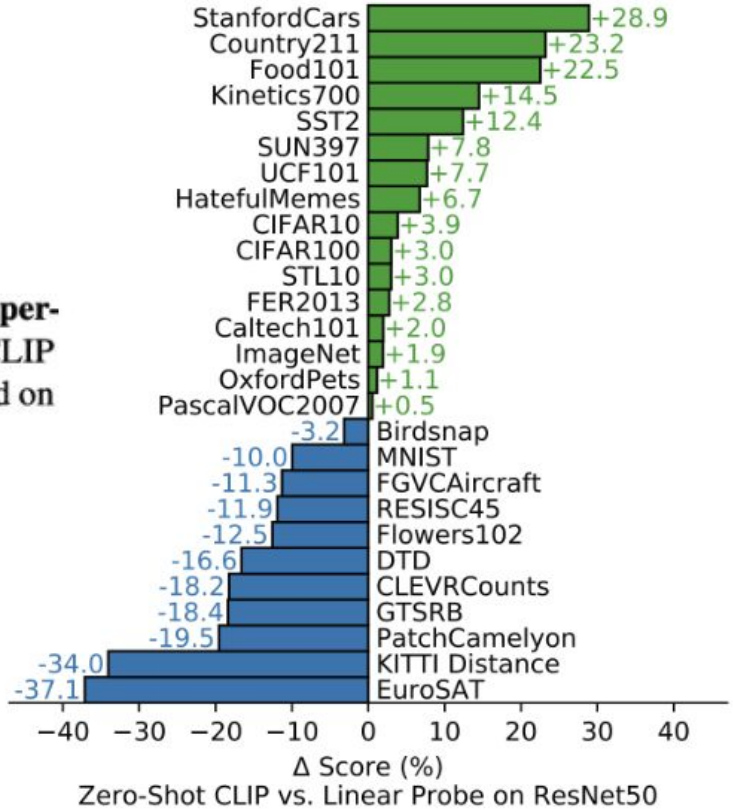


MULTIMODAL REPRESENTATION LEARNING

Contrastive Language–Image Pre-training (CLIP)

Zero-Shot Performance vs. Fully Supervised Baseline

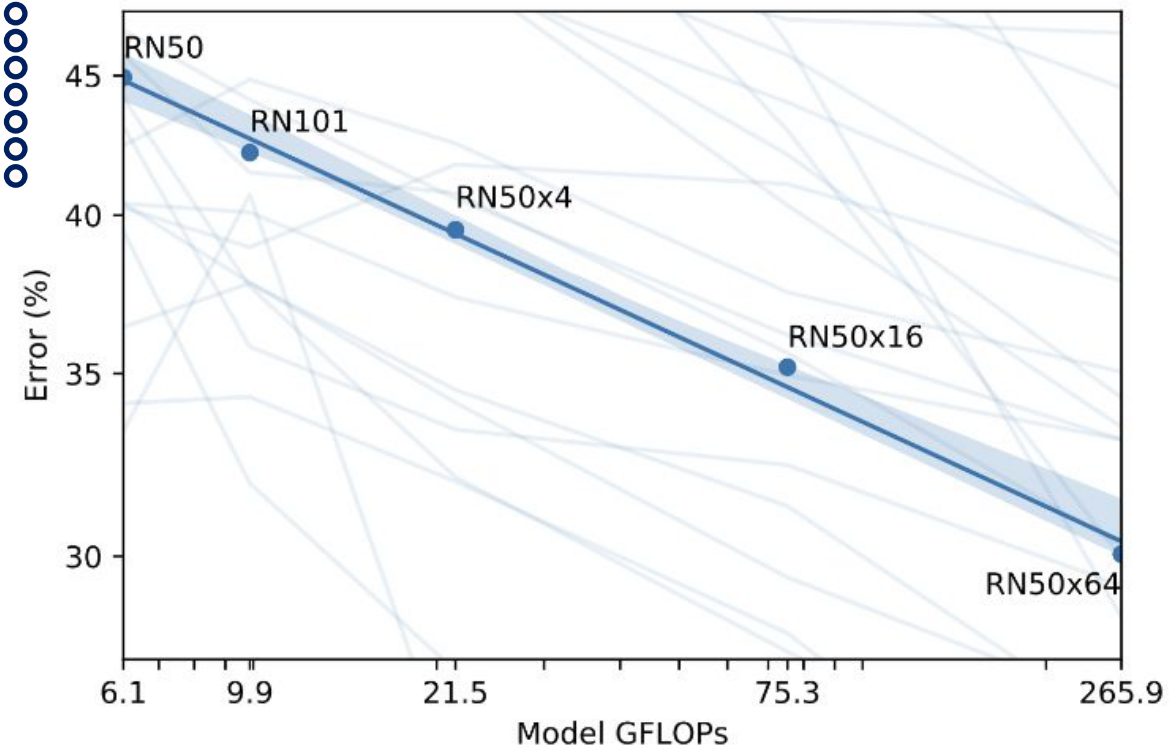
Figure 5. **Zero-shot CLIP is competitive with a fully supervised baseline.** Across a 27 dataset eval suite, a zero-shot CLIP classifier outperforms a fully supervised linear classifier fitted on ResNet-50 features on 16 datasets, including ImageNet.



MULTIMODAL REPRESENTATION LEARNING

Contrastive Language–Image Pre-training (CLIP)

Zero-Shot Performance Scaling as a Function of Pre-training Compute



Key Takeaways

- **Empirical Scaling:** Zero-shot error decreases smoothly as a logarithmic function of pre-training compute (GFLOPS).
- **Model Expansion:** Scaling architectures from RN50 to RN50×64 yields steady performance gains.
- **Predictable Gains:** Increasing compute by >44× reduces top-1 error rate from ~45% down to ~30%.

MULTIMODAL REPRESENTATION LEARNING

Contrastive Language–Image Pre-training (CLIP)

Zero-Shot Classification Code & Execution Example

Input Image



```
==================================================
Cat vs Dog Classification
==================================================

Raw similarity scores (logits):
  a photo of a cat: 18.904
  a photo of a dog: 11.716

Probabilities:
  a photo of a cat: 99.9%
  a photo of a dog: 0.1%

Prediction: a photo of a cat (99.9%)
```

```
============================================================
Zero-shot Classification Results
============================================================

Top 5 predictions:

1. cats on a couch: 89.08%
2. two cats sitting together: 5.39%
3. animals indoors: 1.97%
4. a living room scene: 1.62%
5. pets at home: 1.16%

----------------------------------------

All predictions:

cats on a couch                ████████████████████ 89.08%
two cats sitting together      ▌ 5.39%
animals indoors                 1.97%
a living room scene             1.62%
pets at home                    1.16%
a photo of furniture            0.35%
a photo of a cat                0.33%
a photo of a kitten             0.10%
a photo of a person             0.00%
a photo of a dog                0.00%
```

https://github.com/cmu-l3/anlp-fall2025-code

MULTIMODAL REPRESENTATION LEARNING

Large Language and Vision Assistant (LLaVA)

Visual Instruction Tuning

Haotian Liu[1*], **Chunyuan Li**[2*], **Qingyang Wu**[3], **Yong Jae Lee**[1]
[1]University of Wisconsin–Madison [2]Microsoft Research [3]Columbia University
https://llava-vl.github.io

MULTIMODAL REPRESENTATION LEARNING

Large Language and Vision Assistant (LLaVA)

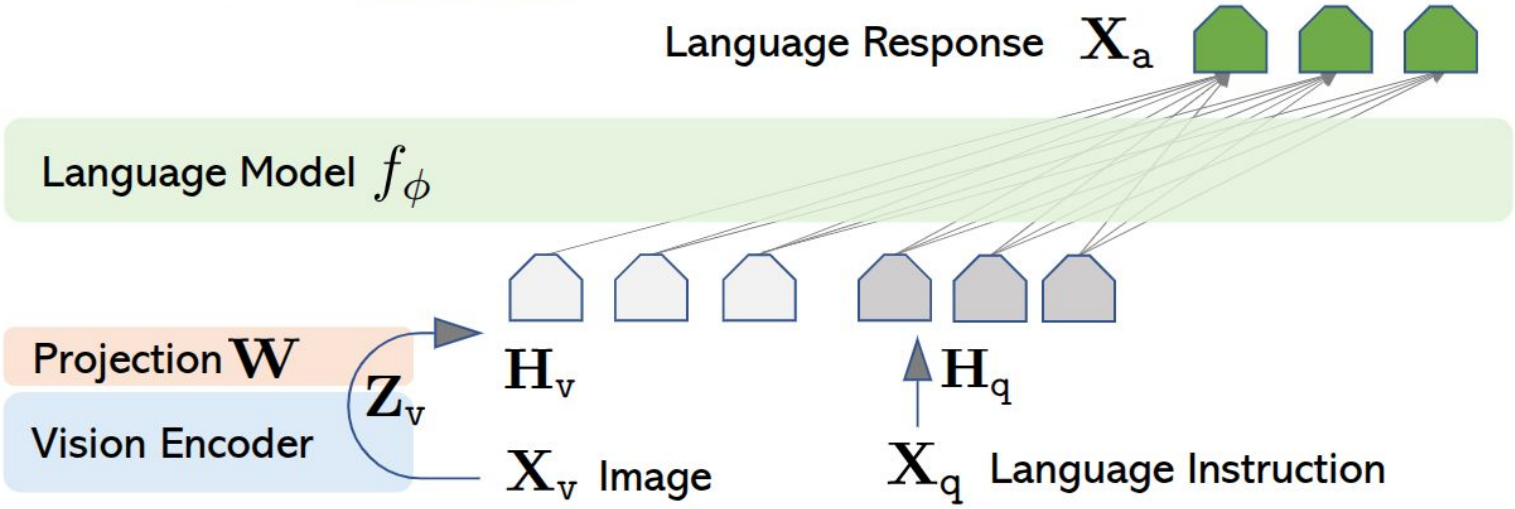


Figure 1: LLaVA network architecture.

MULTIMODAL REPRESENTATION LEARNING

Large Language and Vision Assistant (LLaVA)

Architecture & Multimodal Token Processing Pipeline

Key Architecture Components

Image Tokens (Hv)

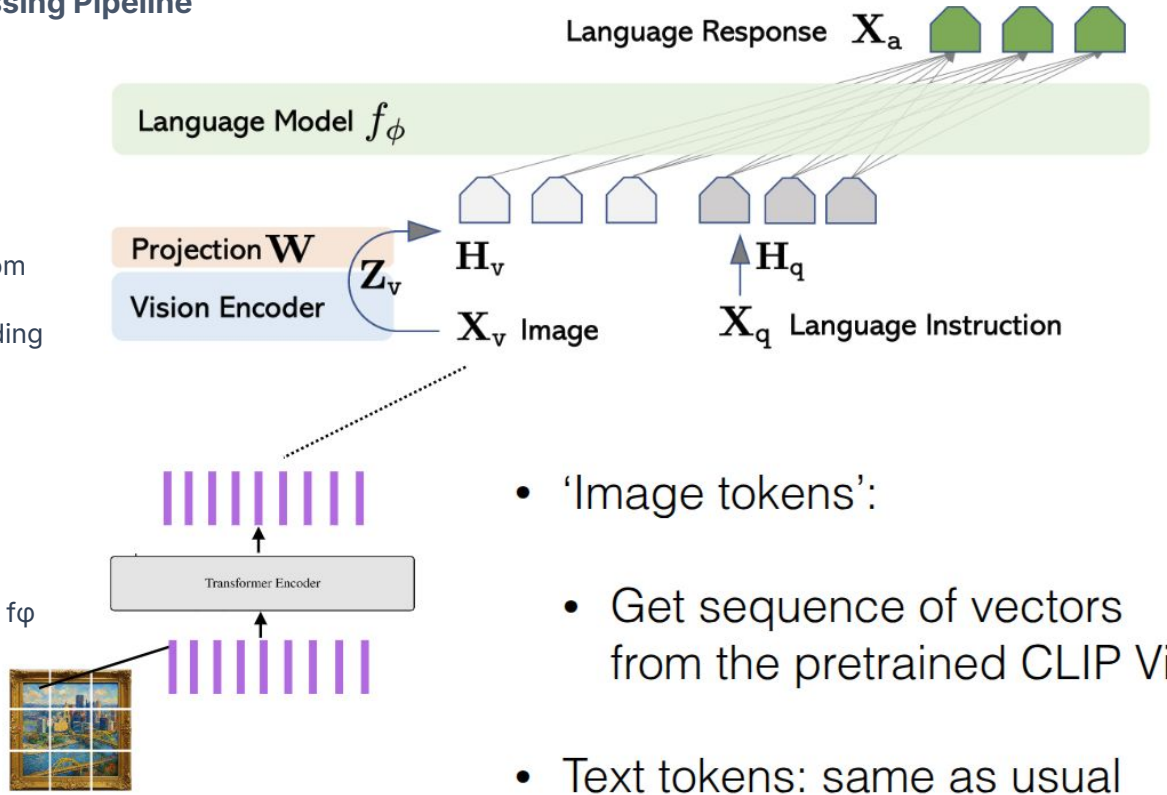
- Extracted as a sequence of feature vectors from pretrained CLIP ViT (Xv)
- Projected via linear matrix W into LLM embedding space (Zv → Hv)

Text Tokens (Hq)

- Processed as standard language instruction tokens (same as usual text inputs)

Language Response (Xa)

- Combined sequence fed into Language Model fφ to generate multimodal output



- 'Image tokens':
  - Get sequence of vectors from the pretrained CLIP ViT
- Text tokens: same as usual

Large Language and Vision Assistant (LLaVA)

Visual Reasoning & Question Answering Example



Source: https://www.barnorama.com/wp-content/uploads/2016/12/03-Confusing-Pictures.jpg

User	What is unusual about this image?
LLaVA	The unusual aspect of this image is a man ironing clothes on the back of a minivan or van. This is not a typical place to perform this activity, as one would usually iron clothes in a more stationary and safe location, such as a home, using a regular ironing board. The scene depicted in the image is peculiar as it involves a makeshift ironing setup on a vehicle, which can be both unsafe and unconventional. Additionally, it is not clear how the man is able to maintain balance and stability while ironing clothes in such an unstable environment.

MULTIMODAL REPRESENTATION LEARNING

MOLMO (AI2): Visual Grounding & Pointing

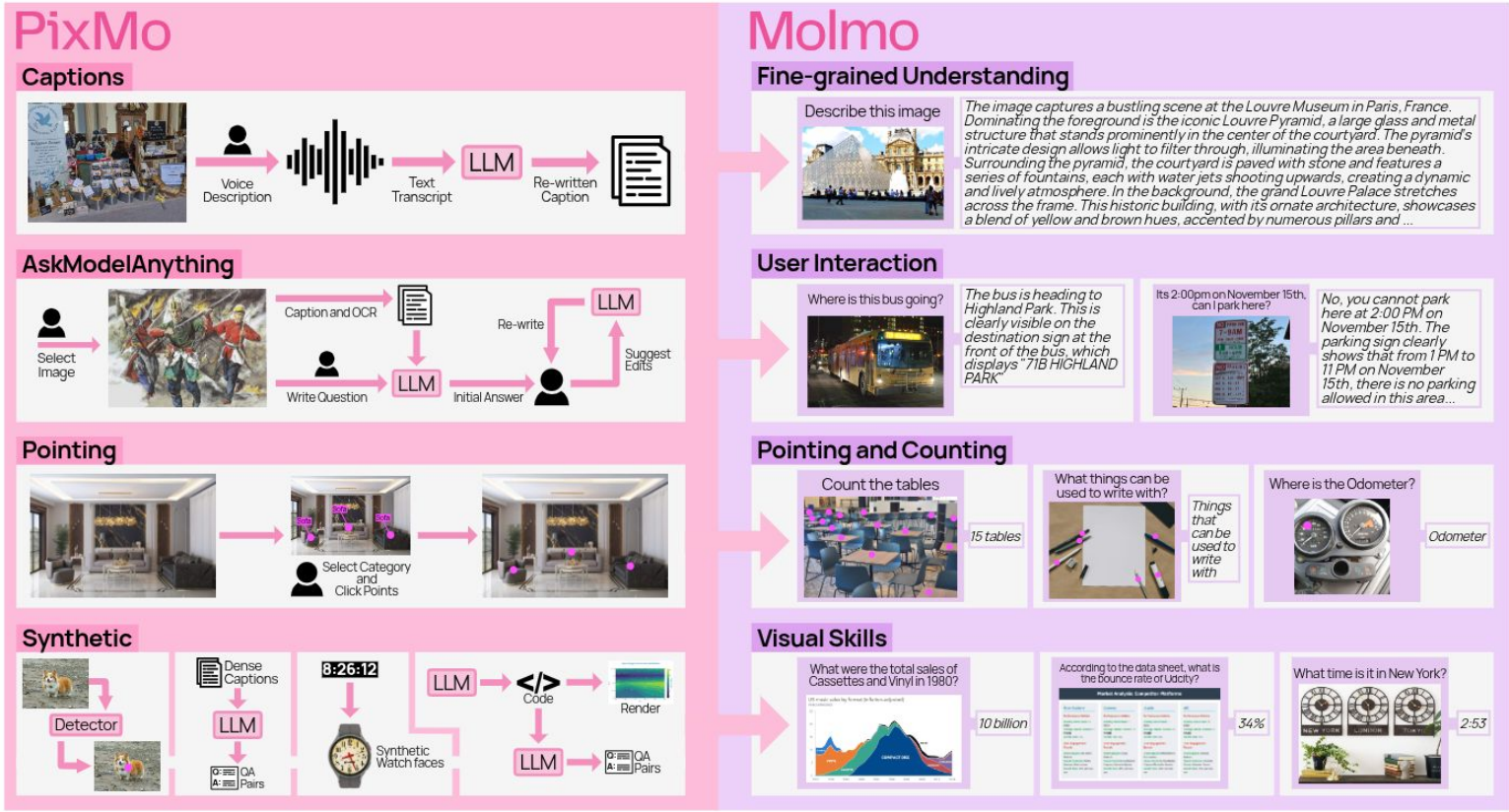
***Molmo and PixMo*:**
Open Weights and Open Data
for State-of-the-Art Vision-Language Models

Matt Deitke*†ψ Christopher Clark*† Sangho Lee† Rohun Tripathi† Yue Yang†
Jae Sung Parkψ Mohammadreza Salehiψ Niklas Muennighoff† Kyle Lo† Luca Soldaini†
Jiasen Lu† Taira Anderson† Erin Bransom† Kiana Ehsani† Huong Ngo†
YenSung Chen† Ajay Patel† Mark Yatskar† Chris Callison-Burch† Andrew Head†
Rose Hendrix† Favyen Bastani† Eli VanderBilt† Nathan Lambert† Yvonne Chou†
Arnavi Chheda† Jenna Sparks† Sam Skjonsberg† Michael Schmitz† Aaron Sarnat†
Byron Bischoff† Pete Walsh† Chris Newell† Piper Wolters† Tanmay Gupta† Kuo-Hao Zeng†
Jon Borchardt† Dirk Groeneveld† Crystal Nam† Sophie Lebrecht† Caitlin Wittlif†
Carissa Schoenick† Oscar Michel† Ranjay Krishna†ψ Luca Weihs†
Noah A. Smith†ψ Hannaneh Hajishirzi†ψ Ross Girshick†ψ Ali Farhadi†ψ Aniruddha Kembhavi†ψ

†Allen Institute for AI ψUniversity of Washington

MULTIMODAL REPRESENTATION LEARNING

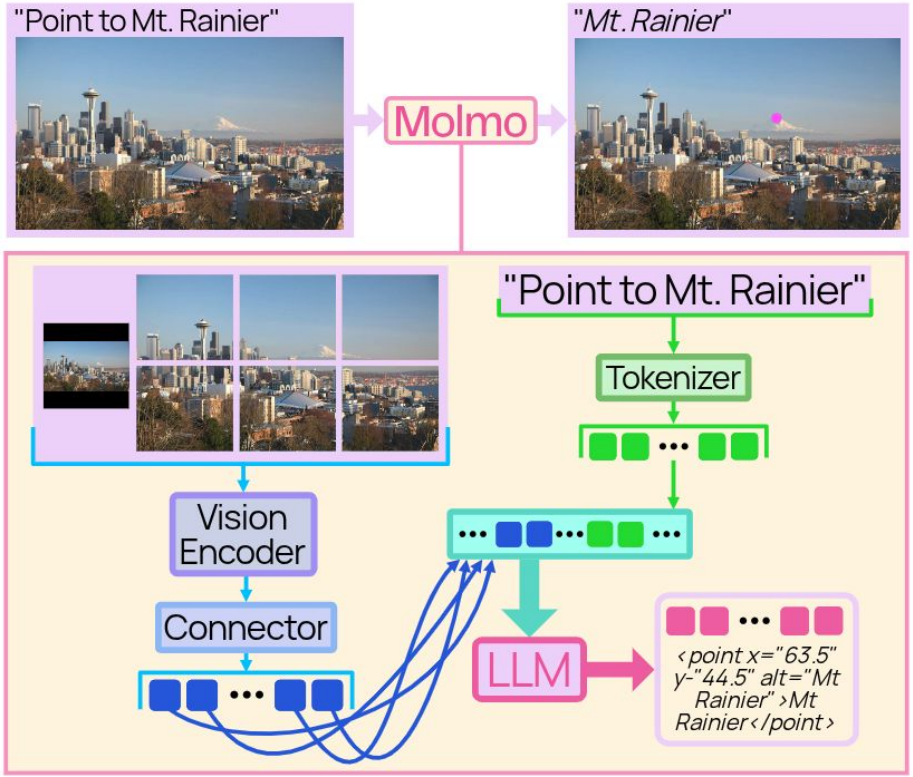
MOLMO (AI2): Visual Grounding & Pointing



MULTIMODAL REPRESENTATION LEARNING

MOLMO (AI2): Visual Grounding & Pointing

End-to-End Architecture for 2D Point Prediction



MULTIMODAL REPRESENTATION LEARNING

MOLMO (AI2): Image Preprocessing

Multi-Crop & Spatial Tokenization Strategy



MULTIMODAL REPRESENTATION LEARNING

MOLMO (AI2): Image Encoding

CLIP ViT-L/14 Patch Pooling & Token Reduction

STEP 01

Base Patch Grid

CLIP ViT-L/14 (336px)

- **Input Size:** 336 × 336 px image
- **Patch Resolution:** 14 × 14 px patches
- **Feature Grid:** 24 × 24 grid

576 Base Patches



STEP 02

Pooling & Projection

LLM Embedding Alignment

- **Patch Pooling:** Pool each 2×2 patch subset
- **Transformation:** Project to LLM's embedding dimension
- **Reduced Grid:** 12 × 12 vectors

144 Tokens / Crop

STEP 03

Multi-Crop Sampling

Global & Local Coverage

- **Global View:** 1 full image representation
- **Local Detail:** 12 high-res image crops
- **Total Execution:** Applied across all 13 views

13 Total Image Views

MULTIMODAL REPRESENTATION LEARNING

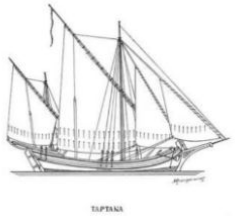
MOLMO (AI2): Detailed Image Captioning

Generating Rich, Highly Detailed Image Descriptions

Key Capabilities

Detailed Caption Generation

- **Fine-Grained Analysis**
  Identifies structural details, materials (wooden boat, sails), and contextual setting.
- **Integrated OCR Recognition**
  Transcribes embedded printed labels ("TAPTANA") and handwritten cursive text.
- **Art & Style Context**
  Recognizes artistic medium (black & white sketch) and notes craftsmanship qualities.



Generate a long caption about this image.

This black and white sketch depicts a boat, elegantly stationed at a pier. The boat, seemingly wooden, showcases a classic design with four prominent sails catching an imagined breeze. Below the boat, text inscribed reads "TAPTANA" in clear, uppercase letters. In the lower right-hand corner, another text, written in cursive and appearing less legible, adds an air of authenticity to the artwork. The meticulous detail in the sails and the overall structure of the boat highlights the craftsmanship and artistry involved in this sketch.

MULTIMODAL REPRESENTATION LEARNING

MOLMO (AI2): 2D Pointing & Counting

Visual Grounding & Spatial Coordinate Generation

Key Capabilities

2D Pointing & Counting

- **Spatial Coordinate Output**
  Generates explicit 2D pixel coordinates (<points x, y>) directly embedded within text responses.
- **Object Counting & Grounding**
  Accurately locates and enumerates multiple object instances (e.g., individual orchestra musicians).
- **Pure LLM Architecture**
  Performs fine-grained visual pointing natively without extra bounding-box or detection heads.



how many musicians.

Counting the <points x1="8.2" y1="37.6" x2="10.9" y2="45.3" x3="13.9" y3="19.4" x4="16.3" y4="11.4" x5="17.4" y5="28.8" x6="23.3" y6="35.5" x7="39.1" y7="27.3" x8="42.4" y8="23.5" x9="56.0" y9="4.8" x10="63.1" y10="28.3" x11="69.3" y11="19.6" x12="71.5" y12="29.7" x13="86.2" y13="17.1" x14="94.6" y14="28....

MULTIMODAL REPRESENTATION LEARNING

MOLMO (AI2): Visual Question Answering

Zero-Shot Visual Reasoning & Fine-Grained QA



What kind of donut is this?

This appears to be a cinnamon sugar donut. The donut itself has a light tan color typical of a traditional fried dough. It's coated with a generous layer of cinnamon sugar, giving it that classic sweet and slightly spicy flavor profile.

The topping isn't cheese as…

Key Capabilities

Visual Question Answering

- **Natural Language QA**
  Answers complex visual queries directly from high-resolution image inputs.
- **Fine-Grained Perception**
  Accurately analyzes subtle textures, ingredients, toppings, and visual nuances.
- **Zero-Shot Reasoning**
  Provides detailed explanations without requiring specialized task-specific tuning.

MULTIMODAL REPRESENTATION LEARNING

MOLMO (AI2): 2D Pointing & Counting

Visual Grounding & Spatial Coordinate Generation

Key Capabilities

2D Pointing & Visual QA

- **Spatial Coordinate Output**

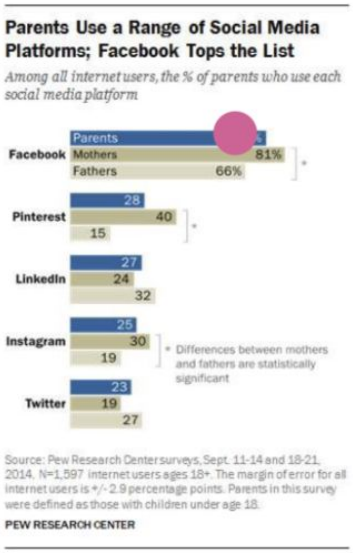
Generates explicit 2D pixel coordinates (<point x, y>) directly embedded within text responses.

- **Visual Question Answering**

Answers complex visual queries while anchoring reasoning to explicit image locations.

- **Pure LLM Architecture**

Performs fine-grained visual pointing natively without extra bounding-box or detection heads.



What percentage of parents use Facebook?

The image shows a breakdown of social media usage among parents. <point x="66.6" y="24.2" alt="Percentage of parents using Facebook">74 percent</point> of parents use Facebook, making it the most popular social media platform among this demographic. This statistic combines both mothers an…

3. Extra

Multi-to-image / Multi-to-Multi

Main Resource For this Section: https://cmu-l3.github.io/anlp-spring2026/

LECTURE OUTLINE

Continuation of Today's Lecture

Core paradigms in generative modeling & vision-language integration

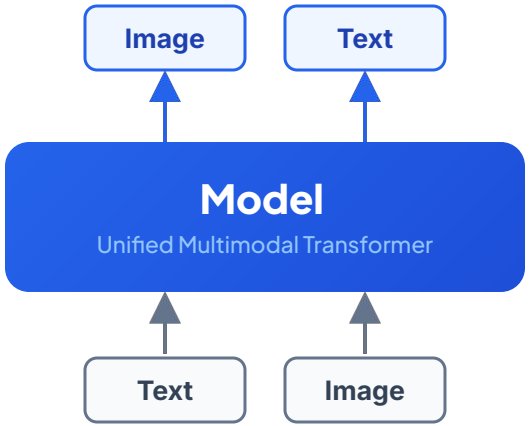
01 **Basic Generative Modeling Paradigms**
Foundational frameworks, VAEs, GANs, and diffusion

02 **Autoregressive Modeling of Pixels**
Sequential pixel prediction dynamics & spatial masking

03 **Autoregressive Modeling of "Image Tokens"**
Discrete visual representations and vector quantization

04 **Other Paradigms**
Masked generative models and non-autoregressive variants

Multimodal Model Architecture



Any-to-Any generation and understanding pipeline

GENERATIVE MODELING

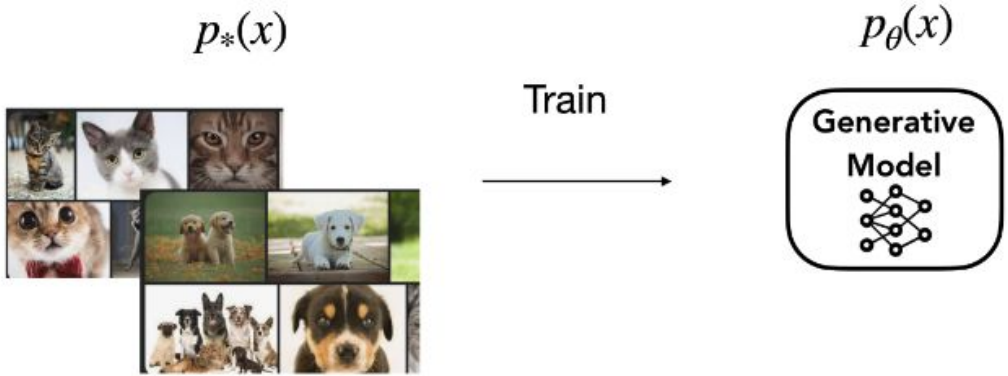
Generative Modeling Paradigms

Core mechanics of learning true data distributions and sampling new data

Training Phase

Data Distribution p*(x) → Model p_θ(x)

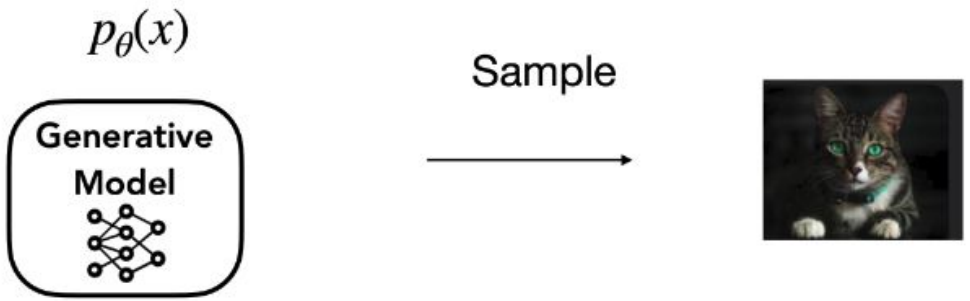
- Fits model parameters θ to training data
- Learns underlying visual features & structure
- Optimizes likelihood or adversarial objectives



Sampling Phase

Model p_θ(x) → Novel Sample x̂

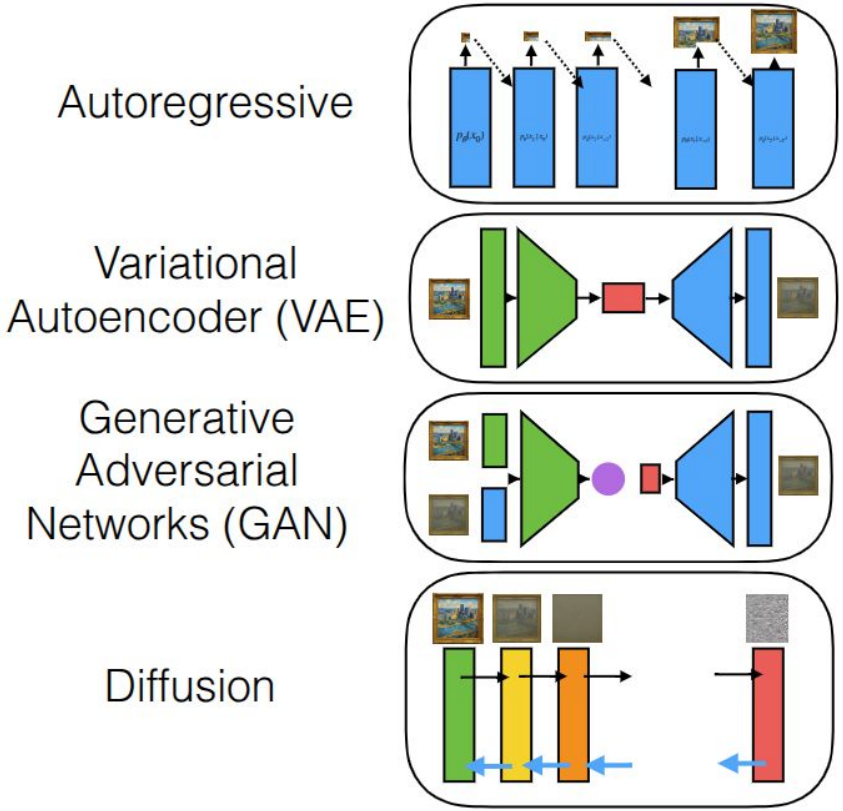
- Queries learned distribution p_θ(x)
- Generates novel, unseen data instances
- Synthesizes high-fidelity visual outputs



GENERATIVE MODELING

Generative Modeling Paradigms

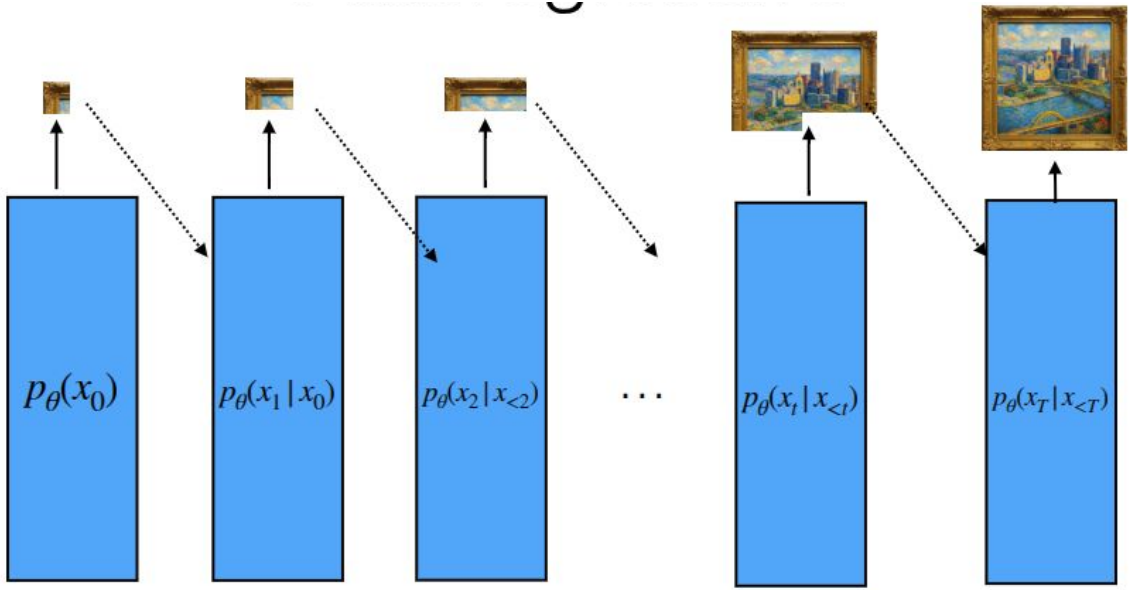
Core mechanics and architectural comparison of fundamental data generation frameworks



GENERATIVE MODELING

Autoregressive Models

Sequential generation conditioned on previously generated tokens or pixels

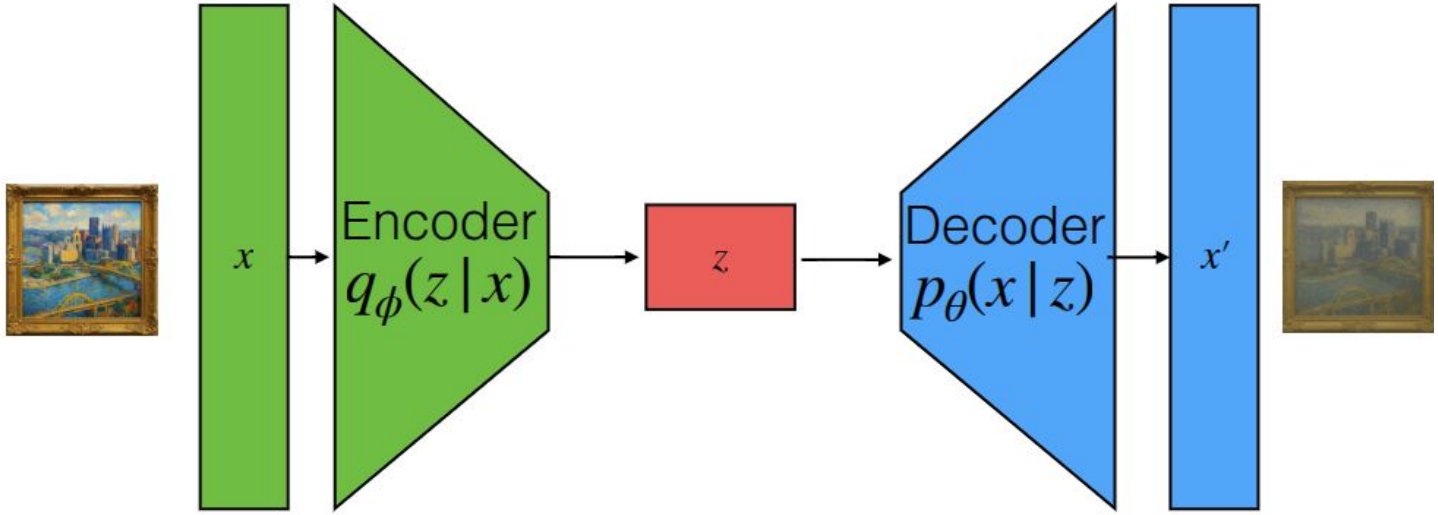


- Generate one dimension (e.g., token, pixel) at a time given the previous ones

GENERATIVE MODELING

Variational Autoencoders (VAE)

Data reconstruction via latent compression and decompression

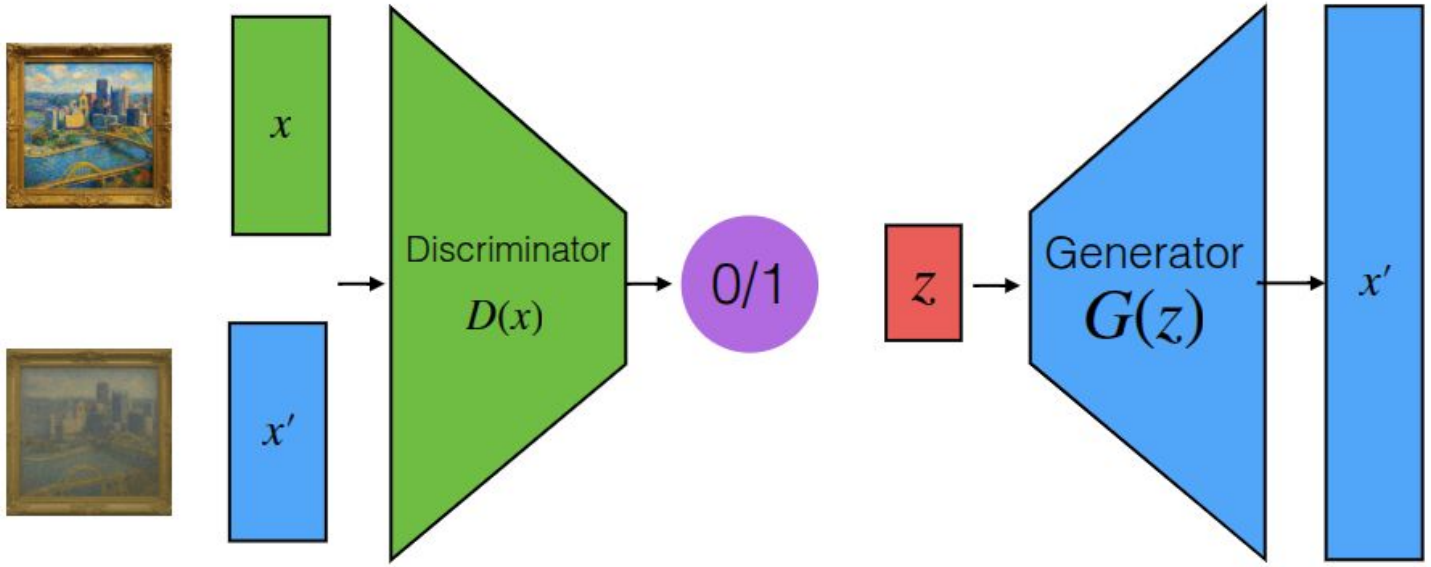


- Learn to reconstruct the image by compressing it into a vector and uncompressing it

GENERATIVE MODELING

Generative Adversarial Networks (GAN)

Adversarial training between a generator and a discriminator

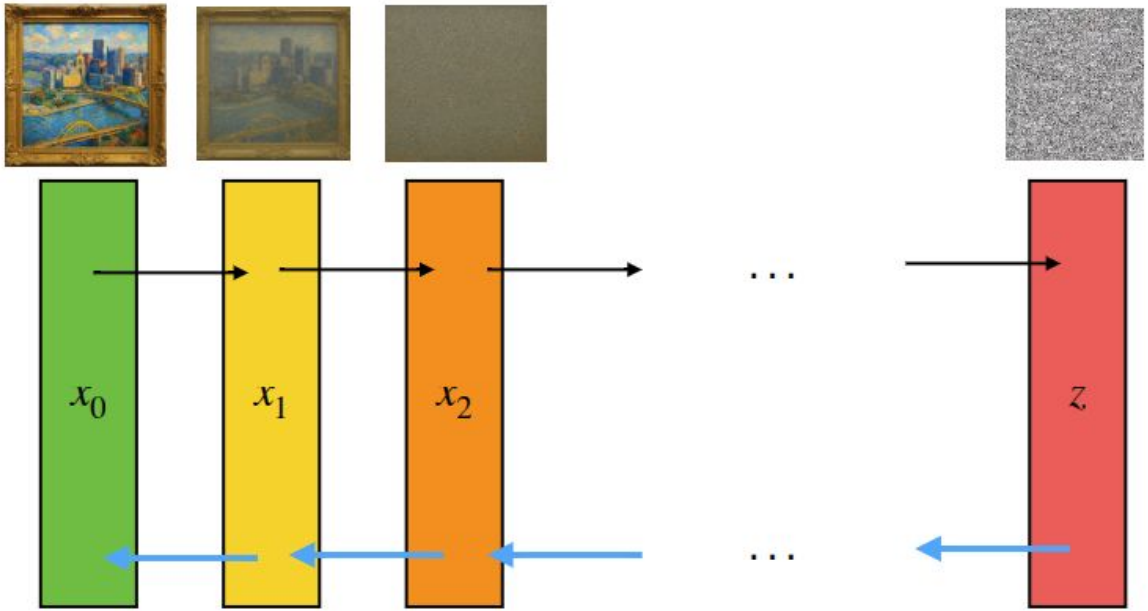


- Learn to generate an image by fooling a discriminator that detects whether an image is fake

GENERATIVE MODELING

Diffusion Models

Gradually adding noise to an image, then learning to de-noise

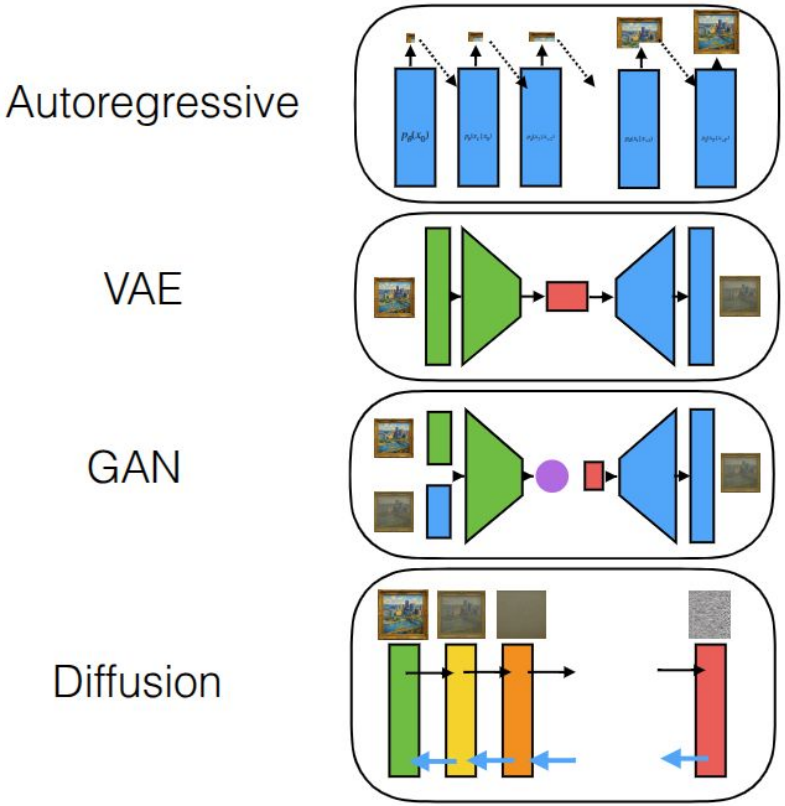


- Gradually add noise to an image, then learn to de-noise

GENERATIVE MODELING

Generative Modeling Paradigms

Core architectural frameworks for visual synthesis & distribution learning

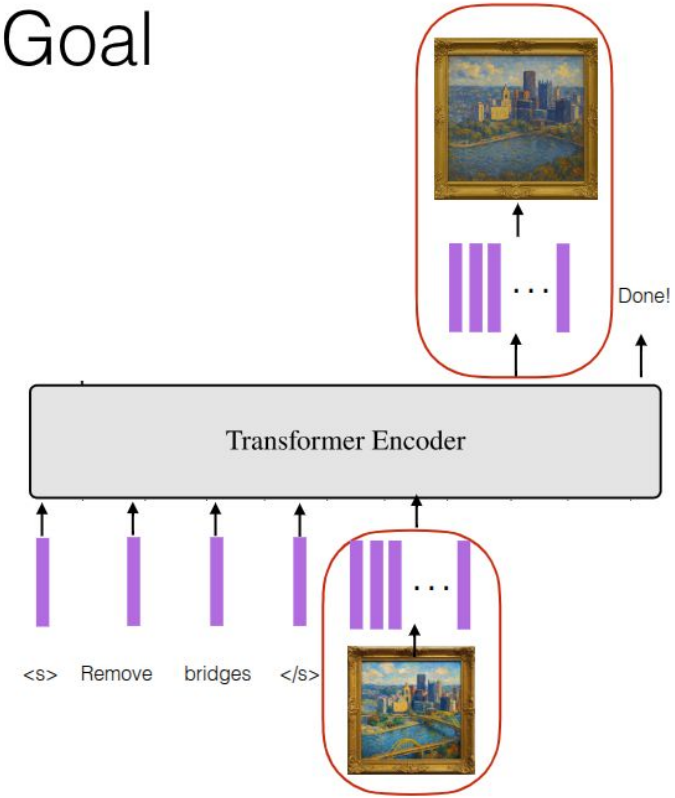


MULTIMODAL TRANSFORMERS

Core Architectural Goal

A unified Transformer framework for joint text and visual token processing

- Image tokenizer
- Image “de-tokenizer”
- Single Transformer for text and image



AUTOREGRESSIVE VISUAL GENERATION

Roadmap & Key Approaches

Evolution from pixel-level autoregression to discrete tokenization

ATTEMPT 1

Standard Auto-Regressive Model

Treat images directly as sequential pixel streams without latent compression.

- **Tokenizer:** Individual pixels act directly as sequence tokens.
- **De-tokenizer:** Generates raw individual pixels sequentially; no standalone de-tokenizer required.

ATTEMPT 2

Discrete Tokenizer / De-tokenizer

Learn compact discrete codebook representations before applying autoregressive models.

- **Vector-quantized VAE (VQ-VAE):** Quantizes visual features into discrete codebook indices.
- **VQ-GAN:** Incorporates adversarial and perceptual losses for high-fidelity reconstruction.

AUTOREGRESSIVE VISUAL GENERATION

Auto-Regressive Models

Treating images as pixel sequences to model joint probability distributions

- **Basic idea**: treat an image as a sequence of pixels
  - Learn a language model over the pixel sequences
- $x_{\text{img}} \in \mathbb{R}^{H \times W \times C} \rightarrow x_1, \ldots, x_T$
  - $x_t \in \{1,2,\ldots,256\}$
- Given a dataset of sequences:
  - $\mathcal{L}_{MLE} = \sum_{t=1}^{T} -\log p_\theta(x_t \mid x_{<t})$

AUTOREGRESSIVE VISUAL GENERATION

Example: PixelRNN [van den Oord et al., 2016]

Modeling explicit pixel-level probability distributions with recurrent neural networks



Figure 1. Image completions sampled from a PixelRNN.

PIXEL-LEVEL MODELING

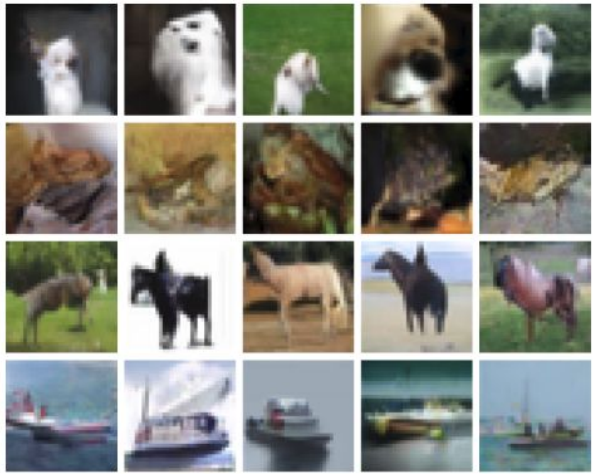
Key Architecture & Mechanics

- **Sequential Dependency:** Models joint image probability by predicting pixels sequentially conditioned on all previous pixels.
- **2D Spatial Context:** Utilizes Row LSTM and Diagonal BiLSTM layers to preserve two-dimensional spatial relationships.
- **Occlusion Completion:** Generates diverse, coherent completions for masked or occluded images by iterative sampling.

AUTOREGRESSIVE VISUAL GENERATION

Example: Image Transformer [Parmar et al., 2018]

Attentive sequence modeling applied to pixel-level image generation





TRANSFORMER ARCHITECTURE

Key Mechanics & Features

- **Self-Attention Dynamics:** Replaces recurrence with local and global self-attention over pixel sequences.
- **Parallel Training:** Significantly increases training throughput compared to sequential RNNs.
- **Fidelity & Scale:** Captures long-range spatial dependencies for crisp visual synthesis and completion.

AUTOREGRESSIVE VISUAL GENERATION

Example: iGPT [Chen et al., 2020]

Generative pre-training from pixels using transformer architectures

PIXEL-LEVEL TRANSFORMER

Key Mechanics & Findings

- **Pre-training Objectives:** Evaluates autoregressive next-pixel prediction and masked pixel prediction (BERT-style) on raw pixel sequences.
- **Representation Learning:** Learns high-quality visual features without supervision, evaluated via linear probes & fine-tuning.
- **Model Scaling:** Larger models (iGPT-L) achieve higher linear probe accuracy with lower validation loss.

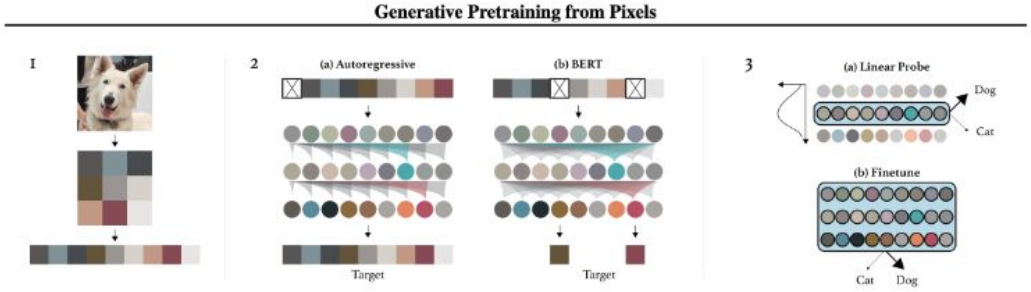
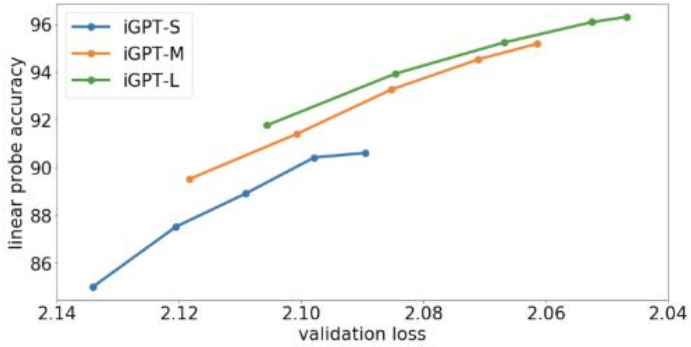


Figure 1. An overview of our approach. First, we pre-process raw images by resizing to a low resolution and reshaping into a 1D sequence. We then chose one of two pre-training objectives, auto-regressive next pixel prediction or masked pixel prediction. Finally, we evaluate the representations learned by these objectives with linear probes or fine-tuning.



AUTOREGRESSIVE VISUAL GENERATION

Auto-regressive Models

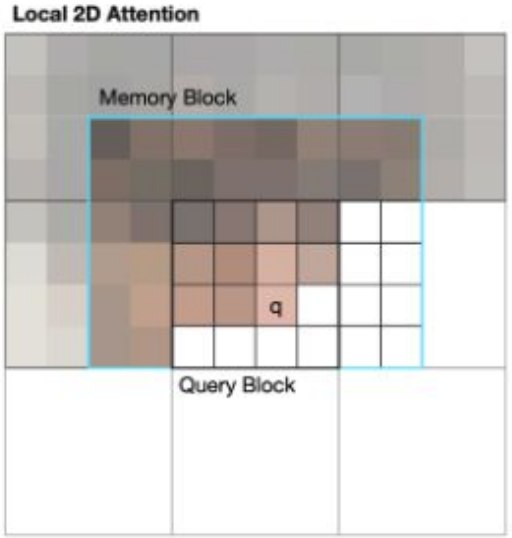
Key challenges in pixel-level autoregressive modeling

PIXEL-LEVEL CHALLENGES

Core Bottlenecks

- **Challenge 1: Sequence Length**
  • 1024 × 1024 image with 3 channels (RGB)
  • Yields ~3 million tokens, creating massive memory overhead for self-attention.

- **Challenge 2: Low Semantic Density**
  • Each pixel carries relatively little semantic information compared to a word token.
  • Requires vast amounts of data to learn spatial structures.



Typically involved alternative memory-efficient attention patterns

AUTOREGRESSIVE VISUAL GENERATION

Auto-Regressive Model Scaling [Yan et al., 2024]

Scaling properties & key findings in next-pixel prediction

SCALING LAWS & FINDINGS

Key Insights

- **Raw Pixels vs. Language Tokens:** Learning on raw pixels requires **10–20× higher** optimal token-to-parameter ratio than language tokens.
- **Task Discrepancy:** Generation quality (Fréchet Distance) requires significantly more training data than classification (Top-1 accuracy).
- **Empirical Verification:** Verified up to **3.5e20 FLOPs**, projecting continued gains with increased compute budget.

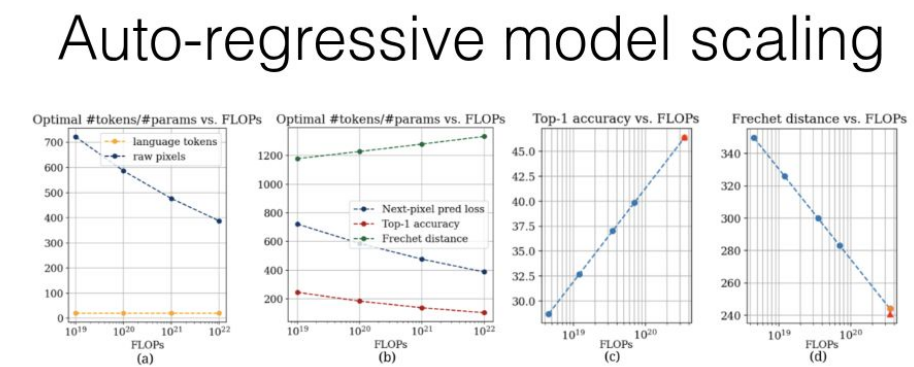


Figure 1. Key findings on the scaling properties of next-pixel prediction, based on training Transformers on 32 × 32 images. (a) Learning on raw pixels (blue line) requires 10 − 20× higher optimal token-to-parameter ratio than learning on language tokens (yellow line). (b) The optimal scaling strategy varies: generation quality (Fréchet Distance, green) requires more training data optimally than classification (Top-1 accuracy, red) or the next-pixel prediction loss (blue). (c)-(d) The optimal-token and optimal-parameter setup is further verified by training models following the scaling prediction. Given 3.5e20 training FLOPs (with 5 × more compute), we project to reach 46.39% accuracy (vs. 46.41% in reality) and 244 Fréchet Distance (vs. 240 Fréchet Distance in reality).

Rethinking generative image pretraining:
How far are we from scaling up next-pixel prediction?

Xinchen Yan [1*] Chen Liang [1*] Lijun Yu [1] Adams Wei Yu [1] Yifeng Lu [1] Quoc V. Le [1]

AUTOREGRESSIVE VISUAL GENERATION

Roadmap

Evolution of approaches for visual generation

ATTEMPT 1

Standard Auto-Regressive Model

Generates images directly at the pixel level sequentially.

- **Pixel-Level Sequence:** High memory overhead due to massive sequence length.
- **Low Information Density:** Individual pixels offer minimal semantic context.

ATTEMPT 2 (PRIMARY FOCUS)

Discrete Tokenizer / De-tokenizer

Learns compact representations of images as sequences of discrete tokens.

Vector-quantized VAE (VQ-VAE)

Compresses images into discrete codebook vectors before sequence modeling.

VQ-GAN

Enhances discrete visual tokenization with adversarial and perceptual objectives.

AUTOREGRESSIVE VISUAL GENERATION

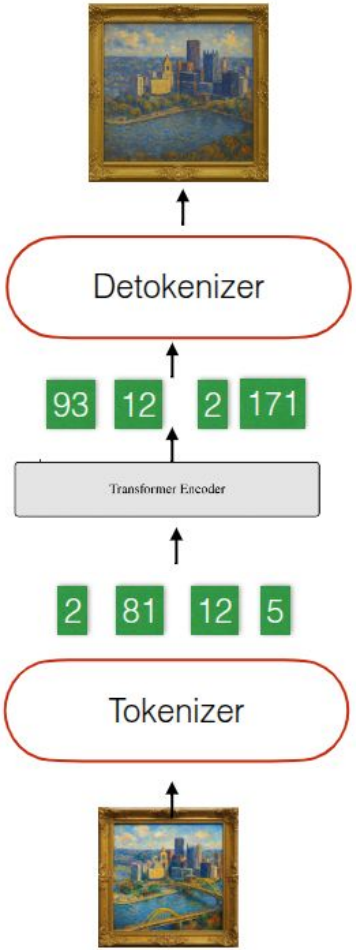
Discrete Visual Tokenization

Representing images as sequences of discrete tokens

CORE CONCEPT

Tokenizer & Detokenizer

- **Compact Representation:** Learn a tokenizer that compactly represents an image as a sequence of discrete tokens.
- **Vocabulary Size:** 8,000 token IDs
- **Sequence Length:** 1,024 tokens per image
- **Core Approach:** VQ-VAE



LATENT VARIABLE MODELS

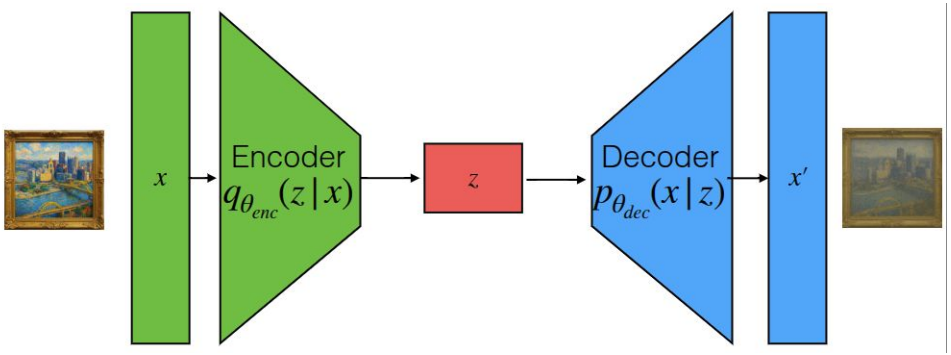
Variational Autoencoder (VAE)

Compressing images into latent vectors and reconstructing them

CORE CONCEPT

Encoder–Decoder Pipeline

- **Latent Compression:** Learns to reconstruct the image by compressing it into a low-dimensional vector z and uncompressing it.
- **Encoder $q\theta(z|x)$:** Maps input image x into a distribution over latent space z.
- **Decoder $p\theta(x|z)$:** Generates reconstructed image x' given latent vector z.



LATENT VARIABLE MODELS

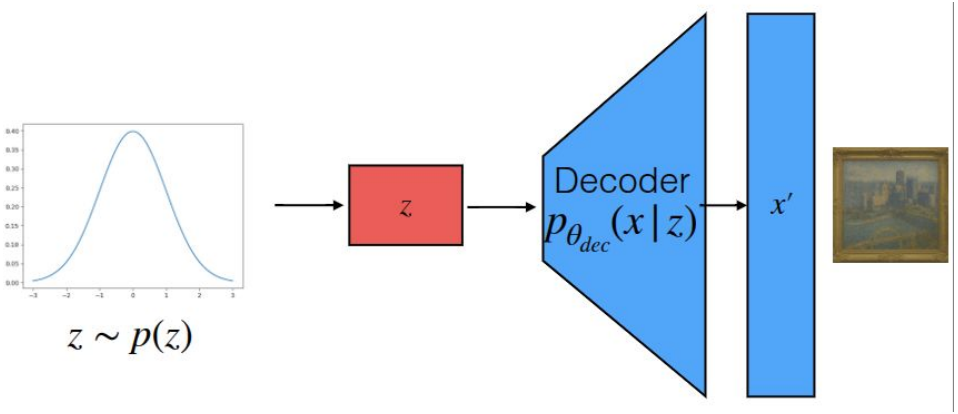
Variational Autoencoder (VAE)

Generating new samples from the latent space after training

GENERATION PROCESS

Latent Sampling & Decoding

- **Latent Prior Sampling:** After training, generate new data by sampling a latent vector z directly from the prior distribution **p(z)**.
- **Decoder Evaluation:** Pass the sampled latent vector z through the decoder network **pθ(x|z)**.
- **Synthetic Reconstruction:** The decoder generates the final output sample x'.



LATENT VARIABLE MODELS

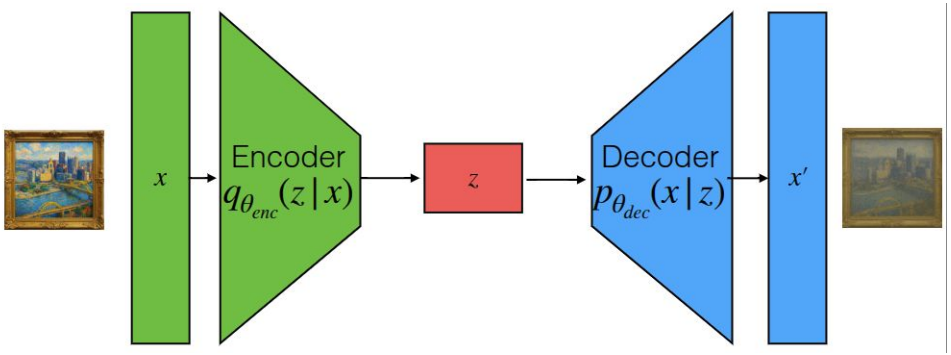
Standard VAE

Kingma & Welling 2013 • Learning low-dimensional representations

CORE CONCEPT & GOAL

Probabilistic Compression

- **Goal:** Learn to compress data into low-dimensional z, then reconstruct or generate new data with it.
- **Generative Assumption:** Data is generated by first sampling $z \sim p(z)$, then decoding into x via $p\theta dec(x|z)$.
- **Encoder Network:** Maps input data x into latent distribution $q\theta enc(z|x)$.



LATENT VARIABLE MODELS

Standard VAE: Training Objectives

Kingma & Welling 2013 • Optimized by balancing two competing goals

GOAL 01 • FIDELITY

Reconstruct the Input Well

Ensures the decoder network accurately reconstructs the original data instance x from the latent vector z.

Reconstruction Loss:

$$-\log p_{\theta_{dec}}(x \mid z)$$

- Measures likelihood of real data under decoder
- Prevents loss of critical input details

GOAL 02 • REGULARIZATION

Keep z Values Well-Behaved

Forces the encoder's output distribution to remain close to a known prior distribution (standard Gaussian).

KL Divergence Regularization:

$$D_{KL}\left(q_{\theta_{enc}}(z \mid x) \| \mathcal{N}(0, I)\right)$$

- Makes latent space smooth and continuous
- Enables smooth interpolation & random sampling

LATENT VARIABLE MODELS

Standard VAE: Reparameterization Trick

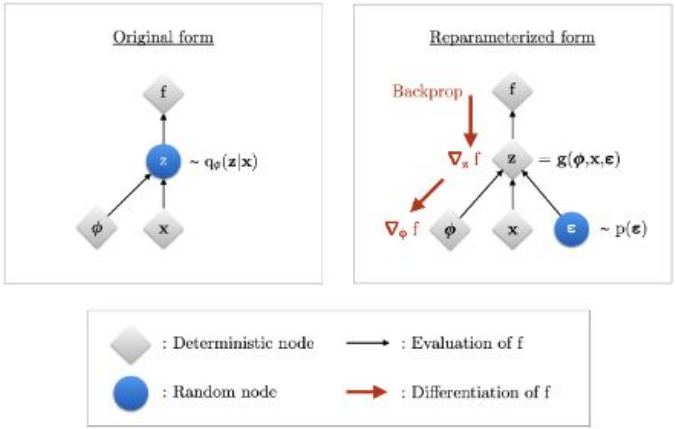
Kingma & Welling 2013 • Enabling backpropagation through stochastic nodes

- Final loss:

$$\mathcal{L}_{VAE}(x) = -\mathbb{E}_{q_{\theta_{enc}}(z|x)}\left[\log p_{\theta_{dec}}(x\,|\,z)\right] + D_{KL}(q_{\theta_{enc}}(z\,|\,x)\|p(z))$$

where $p(z)$ is $\mathcal{N}(0,I)$

"Reparameterization trick"



LATENT VARIABLE MODELS

VAE Loss: Evidence Lower Bound

Mathematical derivation of the Evidence Lower Bound (ELBO) for Variational Autoencoders

$$\log p(x) = \log \int p(x,z)dz$$

$$= \log \int q(z|x)\frac{p(x,z)}{q(z|x)}dz$$

Importance sampling

$$= \log \mathbb{E}_{q(z|x)}\left[\frac{p(x,z)}{q(z|x)}\right]$$

Jensen's inequality

$$\geq \mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z|x)}\right]$$

$$= \mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] + \mathbb{E}_{q(z|x)}\left[\log \frac{p(z)}{q(z|x)}\right]$$

$$= \mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] - \mathrm{D}_{\mathrm{KL}}(q(z|x)\|p(z))$$

$$= -\mathcal{L}_{VAE}$$

Evidence **L**ower **Bo**und ("ELBO")

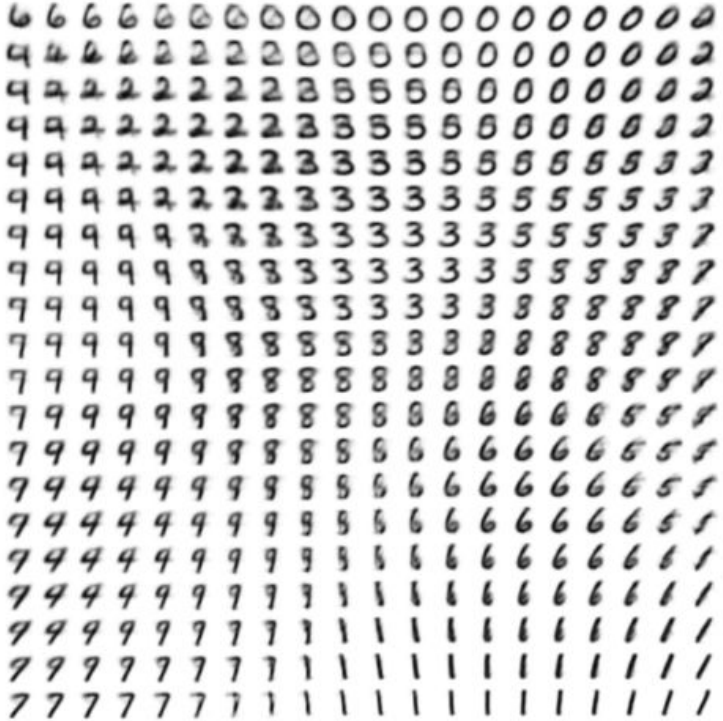
LATENT VARIABLE MODELS

Standard VAE: Learned Manifolds

Kingma & Welling 2013 • Visualizing smooth 2D continuous latent space representations



(a) Learned Frey Face manifold



(b) Learned MNIST manifold

LATENT VARIABLE MODELS

From Continuous to Discrete VAEs

Transitioning from continuous latent vectors to discrete token representations

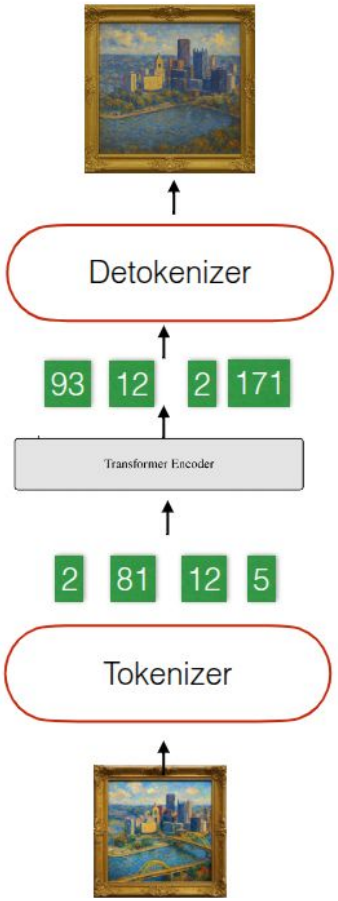
Standard VAEs

Encode and decode images using a **continuous** vector representation.

Discrete VAE (dVAE) Idea

Construct a **discrete** VAE framework to enable:

- **Encoding** an image as a sequence of **discrete tokens**
- **Decoding** an image from a sequence of **discrete tokens**



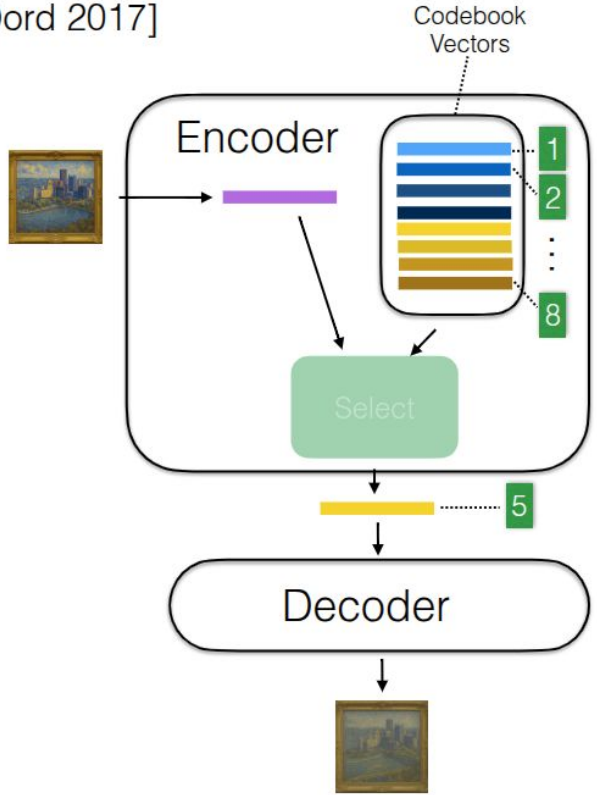
LATENT VARIABLE MODELS

VQ-VAE Architecture

Vector Quantized Variational Autoencoder [van den Oord et al., 2017]

[van den Oord 2017]

- Encoder: $q(z \mid x)$
  - $x \rightarrow z_e(x) \in \mathbb{R}^d$
  - Quantizer: $z_e(x) \rightarrow z_q(x)$
    - Compare $z_e(x)$ to a set of learned codebook vectors
    - $z_q(x) = e_{k^*}$, where $k^* = \arg \min_{j \in \{1,\ldots,K\}} \|z_e(x) - e_j\|_2$
  - $q(z = k \mid x) = \begin{cases} 1 & k = k^* \\ 0 & \text{otherwise} \end{cases}$
- Decoder:
  - $p(x \mid z_q(x))$



LATENT VARIABLE MODELS

VQ-VAE Loss & Optimization

Training objective components and gradient estimation

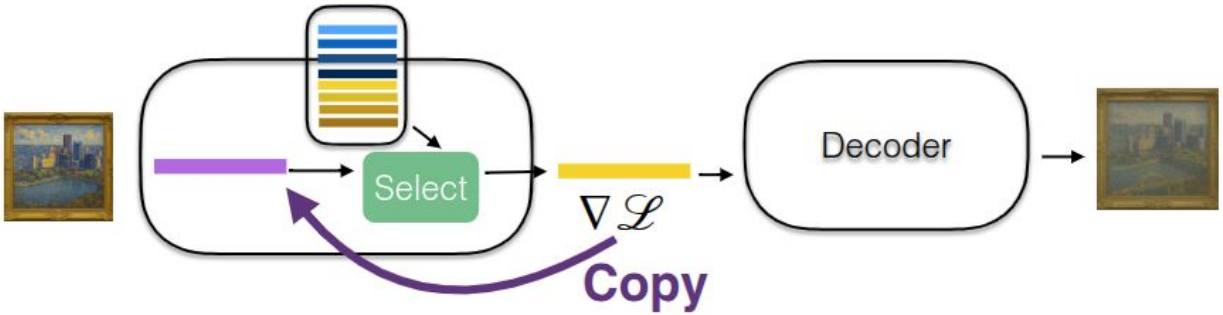
- $\mathscr{L} = \log p(x \mid z_q(x)) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \text{sg}[e]\|_2^2$

  Reconstruction

  Update codebook embedding

  Make encoder "commit to" a codebook embedding

- For reconstruction loss, use a **straight-through gradient estimator** since the codebook lookup is non-differentiable



LATENT VARIABLE MODELS

VQ-VAE Architecture

Vector Quantized Variational Autoencoder pipeline and discrete codebook lookup

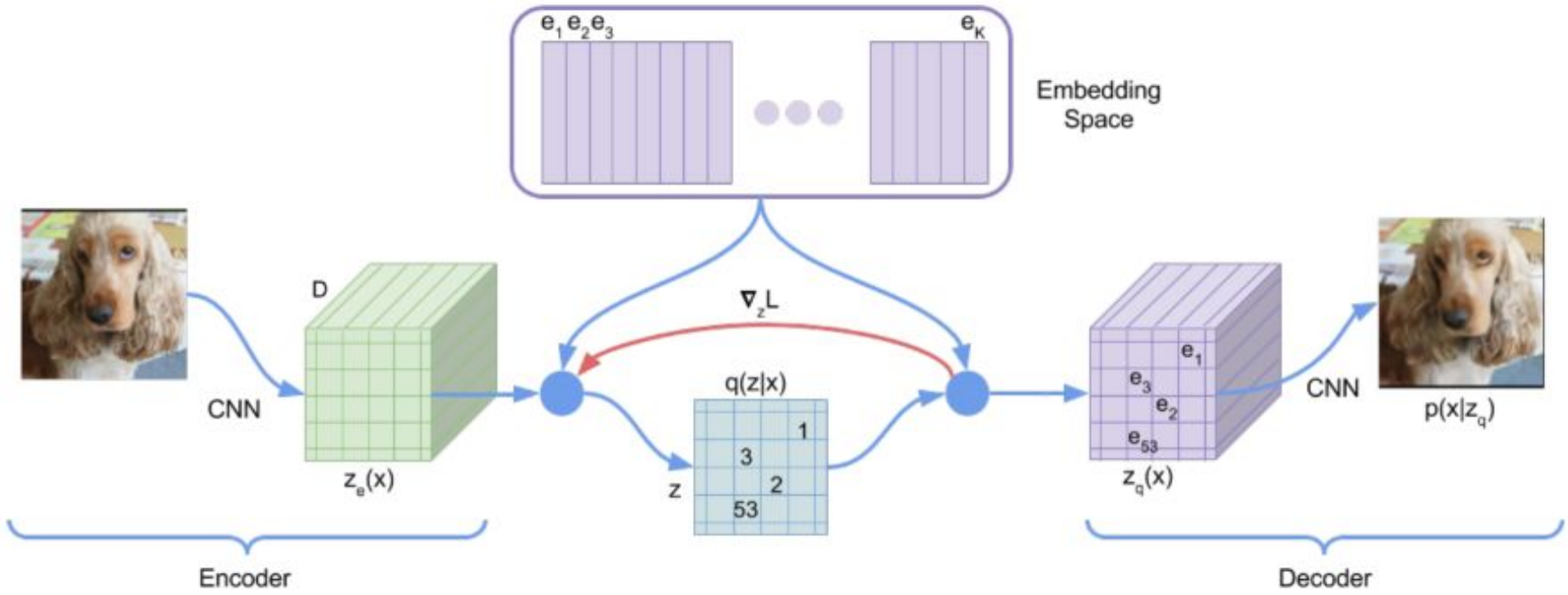
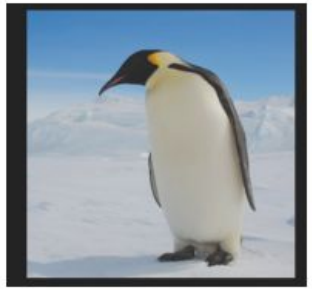


IMAGE TOKENIZATION & RECONSTRUCTION

DALL-E Visual Tokenizer

Discrete latent image tokenization and reconstruction workflow



```
import torch.nn.functional as F

z_logits = enc(x)
z_tokens = torch.argmax(z_logits, axis=1)
z = F.one_hot(z_tokens, num_classes=enc.vocab_size).per
z_tokens, z_tokens.shape
```

✓ 0.5s

```
(tensor([[[7522,  741,  119,  ...,  629, 4695, 4695],
          [ 782, 7459, 4762,  ..., 6913, 5215, 5887],
          [1580, 4066, 5768,  ..., 5677,  224, 2913],
          ...,
          [3990, 7130, 5047,  ..., 6027, 5770, 1212],
          [4898, 5659, 6296,  ..., 5788, 6476, 6397],
          [2757, 3243, 5504,  ..., 5016, 1144, 7261]]]),
 torch.Size([1, 32, 32]))
```

```
x_stats = dec(z).float()
x_rec = unmap_pixels(torch.sigmoid(x_stats[:, :3]))
x_rec = T.ToPILImage(mode='RGB')(x_rec[0])

display_markdown('Reconstructed image:')
display(x_rec)
```

✓ 0.3s

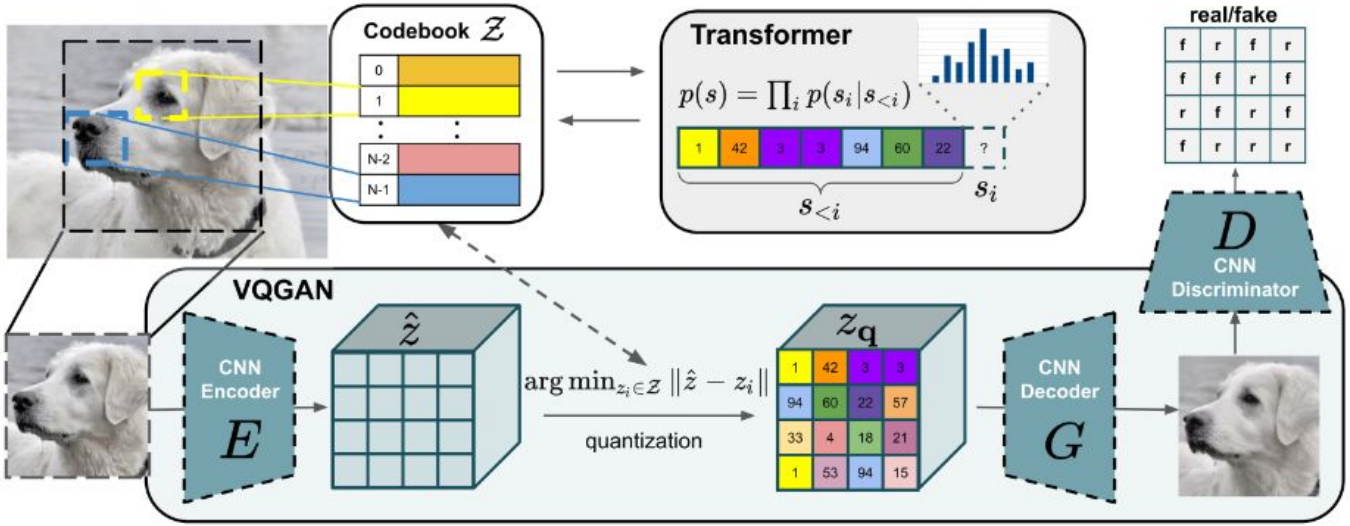


https://github.com/openai/DALL-E

GENERATIVE MODELING & VECTOR QUANTIZATION

VQ-GAN Architecture

Esser et al. 2021 • Combining VQ-VAE and GANs with Transformers



- Modify VQ-VAE objective: use a GAN objective and perceptual loss
- Train an autoregressive transformer ("language model") on the resulting discrete tokenized images. Generate images with it.

GENERATIVE MODELING & VECTOR QUANTIZATION

VQ-GAN: Encoder & Decoder Training

Esser et al. 2021 • Component Roles & Loss Formulations

Training the encoder/decoder:

- $\hat{x} = G(\text{select}(E(x))$
  - G: generator
  - E: encoder
- $\mathcal{L}_{\text{GAN}} = \log D(x) + \log(1 - D(\hat{x}))$
  - D: discriminator

GENERATIVE MODELING & VECTOR QUANTIZATION

VQ-GAN: Transformer LM Training

Esser et al. 2021 • Discrete Latent Sequence Modeling

Training the transformer language model:

- Tokenize lots of images using the tokenizer (encoder) $x_{img} \rightarrow (x_1, x_2, \ldots, x_T)$
- Train an autoregressive transformer:
  - $\mathscr{L} = \mathbb{E}_{x_{img}}[-\log \prod_{t=1}^{T} p(x_t \mid x_{<t})]$
  - Example T: 32 x 32 = 1024
    - Much less than modeling pixels (256x256 = 65,536)!

GENERATIVE MODELING & VECTOR QUANTIZATION

VQ-GAN: Class-Conditional Generation

Esser et al. 2021 • High-Resolution ImageNet Synthesis

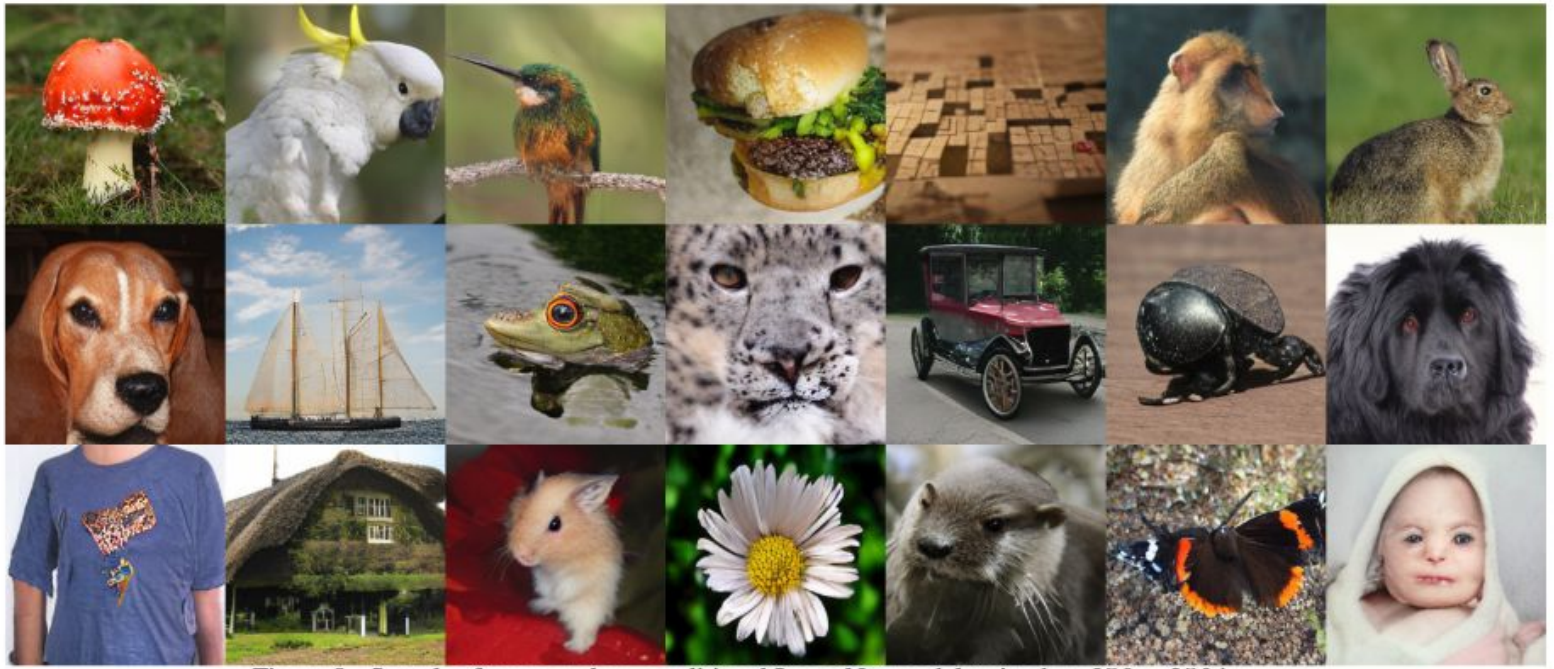
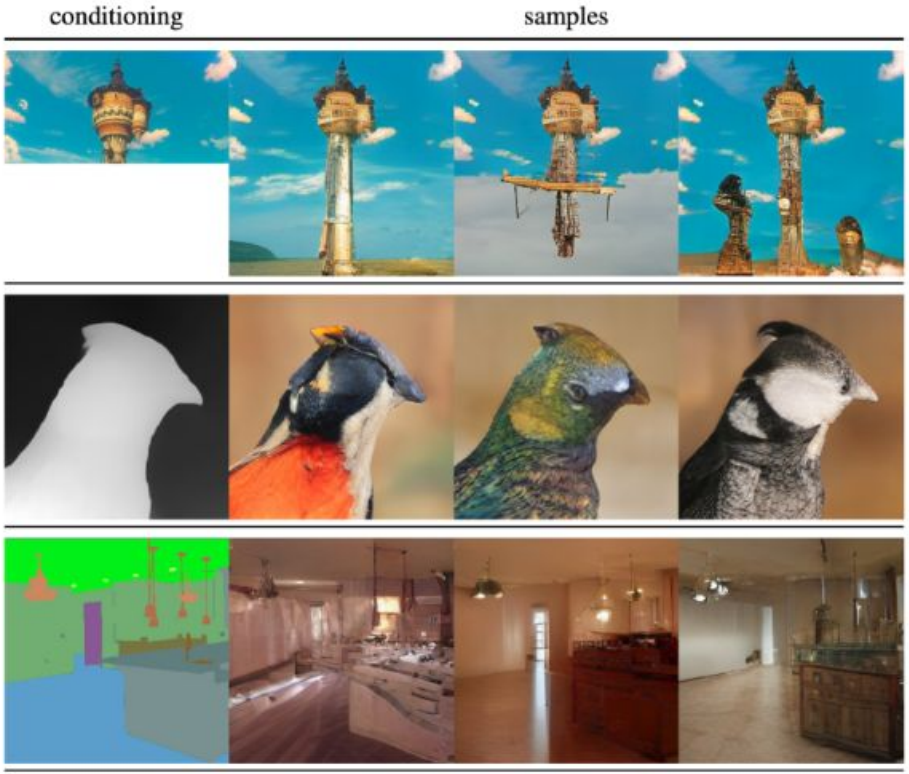


Figure 8. Samples from our class-conditional ImageNet model trained on 256 × 256 images.

GENERATIVE MODELING & VECTOR QUANTIZATION

VQ-GAN: Conditional Image Synthesis

Esser et al. 2021 • Semantic & Depth-Guided Generation



GENERATIVE MODELING & VECTOR QUANTIZATION

Recap: Multimodal Tokenization & Training

Discrete Visual Tokens in Language Models

01 • TOKENIZER

VQ-VAE / VQ-GAN Encoder

Converts continuous image pixels into discrete visual tokens.

02 • DE-TOKENIZER

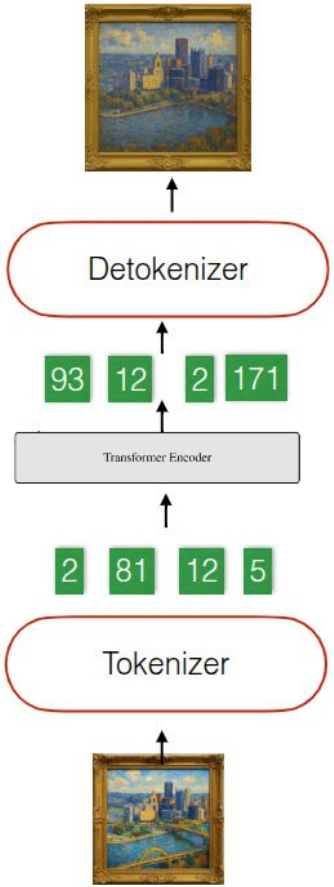
VQ-VAE / VQ-GAN Decoder

Reconstructs output images back from discrete token sequences.

03 • NEXT STEPS

Language Model Integration

- Add visual tokens to language model's vocabulary
- Train / fine-tune model on both text & image tokens



GENERATIVE MODELING & MULTIMODALITY

Example: DALL-E [OpenAI 2021]

Two-Stage Multimodal Image Generation Architecture

- **Stage 1.** We train a discrete variational autoencoder (dVAE)[1] to compress each 256×256 RGB image into a 32 × 32 grid of image tokens, each element of which can assume 8192 possible values. This reduces the context size of the transformer by a factor of 192 without a large degradation in visual quality (see Fig-

- **Stage 2.** We concatenate up to 256 BPE-encoded text tokens with the 32 × 32 = 1024 image tokens, and train an autoregressive transformer to model the joint distribution over the text and image tokens.





(b) an illustration of a baby hedgehog in a christmas sweater walking a dog

GENERATIVE MODELING & MULTIMODALITY

Example: Chameleon [Meta 2024]

Mixed-Modal Early-Fusion Autoregressive Architecture

01 • TOKENIZATION

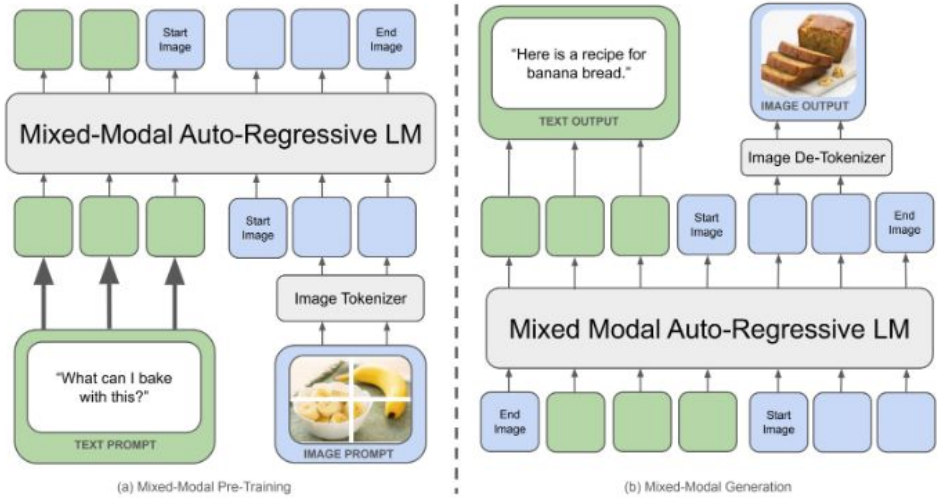
VQ-GAN Tokenizer / De-tokenizer

Train a discrete VQ-GAN image tokenizer and de-tokenizer to convert visual concepts into discrete tokens.

02 • MODEL TRAINING

Language Model Pre-training

Train an autoregressive transformer language model on 10T tokens of mixed text and image representations.



GENERATIVE MODELING & MULTIMODALITY

Example: Chameleon [Meta 2024]

Visual Prompting & Interleaved Generation Example



Prompt: <img> While chameleons are masters of camouflage in the natural world, let's see how they might fare in a man-made setting.
1. Analyze the colors, patterns, and textures in this image and identify the most challenging spot for a chameleon to blend in.
2. Then generate an image of a chameleon with the optimal pattern to camouflage itself.
3. Then, rate and explain how successful each attempt is on a scale from 1-10.

Response: The image is of a stop sign in a forested area, with some sunlight filtering through the trees. The sign has a red octagonal shape with a white border and the word "STOP" written in white letters. In the background, there are green leaves and trees.

Challenging Spot for a Chameleon: The stop sign would be the most challenging spot for a chameleon to blend in due to its bright red color and bold white letters, which would stand out in the forested environment.

Optimal Pattern for Camouflage: To camouflage itself on the stop sign, a chameleon would need to have a pattern that would break up its outline and make it less conspicuous. A pattern with shades of brown and red would help the chameleon blend in more effectively. Additionally, a mottled or dappled pattern would help to scatter light and further reduce the chameleon's visibility.



Rating and Explanation: A rating of 2/10 would be given for the chameleon's ability to blend in on the stop sign. While the chameleon's green and brown pattern helps to break up its outline, the bright red color of the stop sign will still make it stand out in the forested environment.

GENERATIVE MODELING & MULTIMODALITY

Example: DALL-E [OpenAI 2021]

Architecture Pros & Trade-offs

01 • ADVANTAGES

Unified Generative Model

Autoregressive framework where text and visual concepts are treated uniformly as discrete tokens.

02 • LIMITATIONS

Tokenizer Information Loss

Reconstruction quality drop introduced by the discrete image tokenizer and detokenizer.

03 • NEXT PARADIGM

SOTA Image Generation

To produce higher quality, SOTA visual outputs, a different generative paradigm will be explored next.

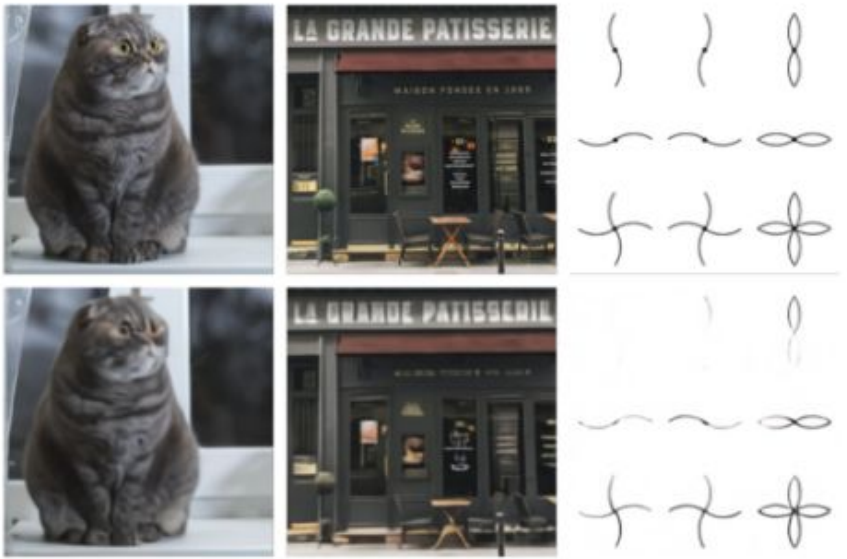


Figure 1. Comparison of original images (top) and reconstructions from the discrete VAE (bottom). The encoder downsamples the

Advanced NLP - Session 14 - Vision-Language Models (VLM: CLIP, LLaVA) & Multimodality - Extra - IUST